《深度学习》实验报告

（第 1 次）

实验题目	基于网格搜索与 K 折交叉验证的 MLP 超参数优化
学号	2024112353
姓名	邴海诺

1. 实验目的

1. **理解核心概念**：理解超参数与模型参数的区别，掌握网格搜索（grid search）与 K 折交叉验证（K-fold cross validation）的原理和流程，理解交叉验证为什么比单次划分更能可靠地估计泛化性能。
2. **掌握实践技能**：使用 PyTorch 构建一个隐藏层大小、丢弃率等超参数可由外部调节的多层感知机（MLP），并搭建“特征工程—交叉验证—网格搜索—结果分析”的完整自动化调优流程。
3. **解决实际问题**：把该流程应用于 SST-2 电影评论情感二分类任务，自动搜索出最佳超参数组合，并评估其相对最差组合带来的性能提升。
4. **评估与分析**：解读调优结果，分析各超参数对模型性能的边际影响，并解释交叉验证分数与独立测试集分数之间差异的来源。

2. 实验内容

2.1 实验任务与数据集

本实验以 SST-2（Stanford Sentiment Treebank）电影评论情感二分类为任务：输入一句英文影评，输出 `negative` 或 `positive`。数据来自课程提供的 `demo_labs/SST-2/`，其中 `train.tsv` 67349 句、`test.tsv` 872 句，均为制表符分隔的 `sentence` 与 `label` 两列。

模型为单隐层 MLP，依次为：稀疏特征输入 → 全连接层 → ReLU → Dropout → 全连接输出层。输入特征由 TF-IDF 从原始句子直接构造，不做人工特征设计。

实验流程为：对 4 个超参数构成的搜索空间，用重复分层 K 折交叉验证逐一评估 36 个超参数组合，按平均验证准确率排序选出最佳组合，最后在完整训练集上重训并在测试集上评估。

2.2 相对示例程序的新增功能

课程提供的示例程序 `demo_labs/lab1-hyper-paramter.py` 给出了本实验的基本方向：在 Iris 数据集上用 MindSpore 实现 MLP，对 2 个超参数做网格搜索与 5 折交叉验证。本实验沿用了该方向，但在框架、任务和数据规模上都做了实质扩展，共新增以下 9 项功能，其中前 3 项为核心新增。

1. **折内拟合的特征流水线（修复数据泄露）**：示例在 `KFold` 划分之前对全部数据 `fit_transform` 标准化，验证折的统计量会泄漏进训练折。本实验把 TF-IDF 的词表与 IDF 全部放进每折训练集内部拟合，再用该折学到的词表变换验证折。为控制开销，每折的特征只拟合一次，由所有超参数组合复用，避免 $36 \times 10 = 360$ 次重复计算。
2. **搜索空间由 2 个超参数扩展到 4 个**：示例只搜索 `hidden_size` 与 `dropout`。本实验增加 `learning_rate` 与 `weight_decay`，组合数由 4 增至 $3 \times 3 \times 2 \times 2 = 36$，能够观察四者的交互作用。
3. **交叉验证由单次 K 折升级为重复分层 K 折**：示例为单次 5 折。本实验采用 `RepeatedStratifiedKFold` 做 5 折 $\times$ 2 次重复共 10 个划分，分层保证每折类别比例与总体一致，并同时报告平均准确率与各折标准差，降低单次划分偶然性带来的误判。
4. **早停与学习率调度**：示例固定训练 100 轮且无验证集参与。本实验加入 `ReduceLROnPlateau`（factor 0.5、patience 2）与基于验证损失的早停（patience 5），并恢复最优权重。早停所用的验证集从每折训练集内部再切出 10%，与外层用于评分的验证折严格分离，避免用评估折选择训练轮次。
5. **评估指标由单一准确率扩展为四项指标加混淆矩阵**：示例只报告 `accuracy_score`。本实验同时计算 Accuracy、宏平均 Precision、Recall、F1，并在最终测试集上给出混淆矩阵，便于观察两类是否存在系统性偏向。

6. **增加独立测试集与最终重训**：示例仅在交叉验证内部循环，没有留出测试集。本实验以交叉验证各折最优轮次的中位数作为固定轮数，在完整 67349 句训练集上重训最佳组合，在 872 句测试集上评估，并保存模型权重。
7. **joblib 并行搜索与串行/并行耗时对比**：示例为串行的 `for` 循环。本实验使用 joblib 按超参数组合并行分发训练任务，并实测串行与并行的墙钟耗时与加速比，同时分析该加速比受什么因素制约。
8. **结果可视化与落盘**：示例没有任何绘图。本实验输出 6 张图（组合排名及误差棒、Top-12 箱线图、hidden_size × dropout 热力图、学习率与权重衰减的边际影响、最佳组合损失曲线、测试集混淆矩阵），并把交叉验证汇总与逐折结果写成 CSV、把最佳超参数与测试指标写成 JSON。
9. **稀疏输入格式与批量大小的实现级优化**：示例用稠密小特征，不涉及稀疏输入与显存开销。本实验改用稀疏 TF-IDF 直接接入后实测发现，CUDA 上 `torch.sparse.mm` 的开销按「次」计而非按数据量计，第一层 50000 × hidden 的参数更新成了主导开销且与批次大小无关。据此把每批张量格式由 COO 换成 CSR（同批量下快 1.3 倍）、把 batch size 由 256 提到 2048，单轮训练由 1425.0 ms 降到 149.2 ms（18.7 倍），并单独验证了放大批量不损失准确率。量化过程与机制分析见 4.6 节。

此外还做了一项消融对照：把“TF-IDF 先经 SVD 降维、再喂稠密模型”这条常见路线与本实验的稀疏直接接入放在一起比较，前者在本任务上损失显著，本实验据测试集结果改用稀疏输入的直接全连接层，相关数据见 4.5 节。

3. 实验环境

- 操作系统：Linux（CachyOS，内核 7.2.4）
- Python：3.12.12（uv 管理的项目虚拟环境 `.venv`）
- 深度学习框架：PyTorch 2.11.0+cu128
- 其他依赖：scikit-learn 1.9.1、numpy 2.5.3、matplotlib 3.11.2、joblib（随 scikit-learn 安装）
- 硬件：Intel Core i9-14900HX（24 物理核 / 32 逻辑核）、NVIDIA GeForce RTX 4060 Laptop GPU（8 GB 显存）、内存满足全部折内特征常驻
- 计算设备：训练在 CUDA 上完成，随机种子固定为 42，全部结果可由同一条命令复现

4. 实验过程、结果及分析

4.1 理论基础与总体思路

超参数与网格搜索。模型参数（权重、偏置）由优化算法从数据中学得，而超参数（隐藏层大小、丢弃率、学习率、权重衰减系数等）必须在训练开始前由人指定，且无法通过梯度下降求得。网格搜索是最直接的调参策略：给定每个超参数的候选集合，取笛卡尔积得到全部组合，逐一训练并评估，最后选出表现最好的组合。其优点是实现简单、易于并行、结果可复现；缺点是代价随超参数个数呈指数增长，因此实际使用时需要控制搜索空间的规模。

K 折交叉验证。若只用一次训练/验证划分来评价超参数，评估结果会受该次划分的偶然性影响：某组超参数可能恰好被分到“好带”的验证集上。K 折交叉验证把数据分成 K 份，轮流以其中一份作验证集、其余 K-1 份作训练集，得到 K 个分数并取平均，从而显著降低划分偶然性；同时每个样本都恰好参与一次验证，样本利用更充分。本实验进一步采用重复 K 折（重复 R 次、每次重新洗牌划分），得到 K × R 个分数，用均值衡量性能、用标准差衡量稳定性。

需要强调的是，交叉验证只能估计模型在**与训练数据同分布**的数据上的性能。若测试集分布与训练集不同，交叉验证分数会系统性偏高——这一点在本实验中确实出现了，详见 4.5 节。

防泄露原则。任何依赖数据统计量的预处理（本实验中的 TF-IDF 词表与 IDF）都必须在训练折内部拟合，否则验证折的信息会通过预处理渗入训练过程，使交叉验证分数被高估。这是本实验相对示例程序的首要修正。

总体思路。整体流程为：读入 SST-2 → 构造重复分层 K 折划分，并在每折内部拟合 TF-IDF 得到稀疏特征 → 对 36 个超参数组合分别跑完 10 折，每折内用早停确定轮次并在外层验证折上评分 → 按平均准确率排序，选出最佳组合 → 用最佳组合在完整训练集上按固定轮数重训，并在测试集上评估。

4.2 数据准备与特征工程

数据规模与标签分布。训练集 67349 句中负例 29780 句、正例 37569 句（正例约占 55.8%）；测试集 872 句中负例 428 句、正例 444 句（基本均衡）。训练集句子平均 9.4 个词（最短 1、最长 52），测试集句子平均 19.5 个词（最短 2、最长 47）。**测试集句子长度约为训练集的两倍**，这一分布差异是 4.5 节讨论交叉验证与测试集分数落差的关键。

特征构造。把每句文本转为 TF-IDF 稀疏向量，参数为：1-gram 与 2-gram 并用（`ngram_range=(1, 2)`）、`min_df=3`、`max_features=50000`、启用 `sublinear_tf`。每句平均约 14 个非零特征。

划分方式。采用 `RepeatedStratifiedKFold`，5 折 × 2 次重复 = 10 个划分，每折约 48491 句用于训练、13470 句用于验证，分层保证各折正负比例一致。每折的训练集内部再按 9:1 分层切出一份“早停验证集”（约 5388 句），仅供早停与学习率调度使用；每折的 10 个划分互不重叠的验证折才用于最终评分。

稀疏特征直接接入网络。由于每句仅有约 14 个非零特征，无须先降维成稠密向量，直接把稀疏的 TF-IDF 矩阵喂给网络第一层，用 `torch.sparse.mm` 完成稀疏矩阵乘法。这样既保留了全部 50000 维词表信息，也把每折的特征构建耗时压到 0.5 秒以内。

4.3 模型与关键实现

模型结构。单隐层 MLP，前向传播为：稀疏 TF-IDF 输入 → `Linear(input_dim, hidden_size)` → `ReLU` → `Dropout(p)` → `Linear(hidden_size, 2)`。第一层以稀疏矩阵乘法实现，因此无需把输入稠密化。`hidden_size` 与 `dropout` 在实例化时由外部传入，供网格搜索遍历。

损失与优化。损失函数为交叉熵（`nn.CrossEntropyLoss`），优化器为 Adam，学习率与权重衰减由搜索空间给出。

训练策略。小批量训练，batch size 2048（取值依据见 4.6 节），搭配两项调度：`ReduceLROnPlateau` 在验证损失停滞 2 轮时把学习率减半（下限 1e-5）；早停在验证损失连续 5 轮无改善时终止训练，并回滚到历史最优权重。最大轮数设为 30。由于每折的早停验证集与外层评估折互不重叠，用早停选择轮次不会污染评估结果。

结果落盘。脚本把交叉验证汇总写为 `results/cv_summary.csv`、逐折明细写为 `results/cv_per_fold.csv`、最佳超参数与测试指标写为 `results/best_params.json`、最佳模型权重写为 `results/best_model.pt`。

可复现性。全程固定随机种子 42：折划分由 `random_state=42` 决定，每折训练前以 `torch.manual_seed(42 + fold)`重置随机源，每轮打乱顺序由 `np.random.default_rng(42 + epoch)`生成。已实测两次独立运行得到的逐组合平均准确率逐位一致。

4.4 训练过程与运行结果

运行方式。全部结果由同一条命令产生，`--bench-combos 8` 表示先用搜索空间里的前 8 个组合做一次串行/并行耗时对比，再执行完整网格搜索：

```
.venv/bin/python -u lab1/hyperparameter_search.py --jobs 4 --torch-threads 2 --bench-combos 8
```

流程耗时。构建 10 个折划分与折内 TF-IDF 特征共 5.41 s（每折 0.5–0.6 s）；36 个组合 × 10 折 = 360 次训练，并行（n_jobs=4）墙钟 602.22 s，累计训练耗时 2296.7 s，平均并发度 3.81；程序总耗时 676.36 s。

搜索结果。图 4-1 给出全部 36 个组合按平均验证准确率的升序排列与各折标准差，图 4-2 是前 12 名的箱线图（展示 10 折分数的离散程度）。表 4-1 摘出前三名与后三名。

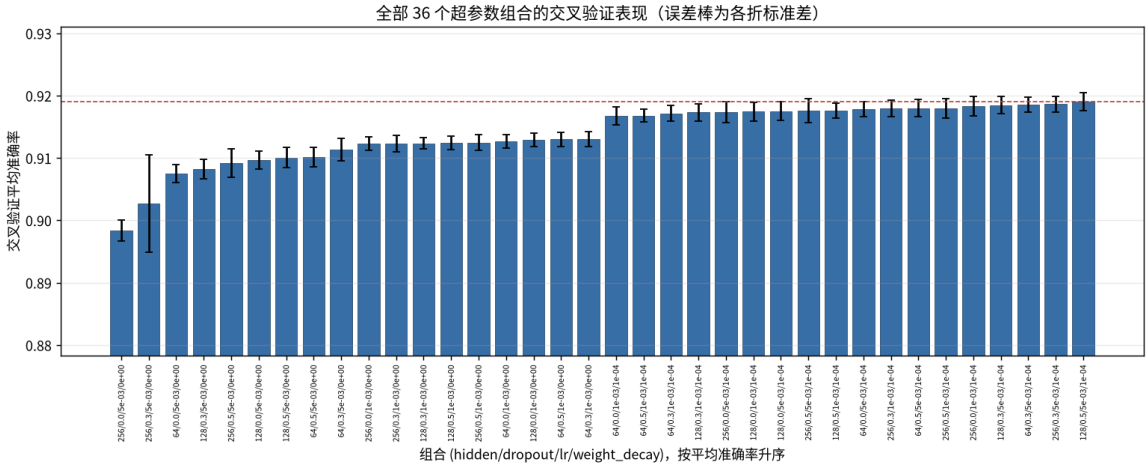


图 4-1　全部 36 个超参数组合的交叉验证平均准确率（误差棒为 10 折标准差）

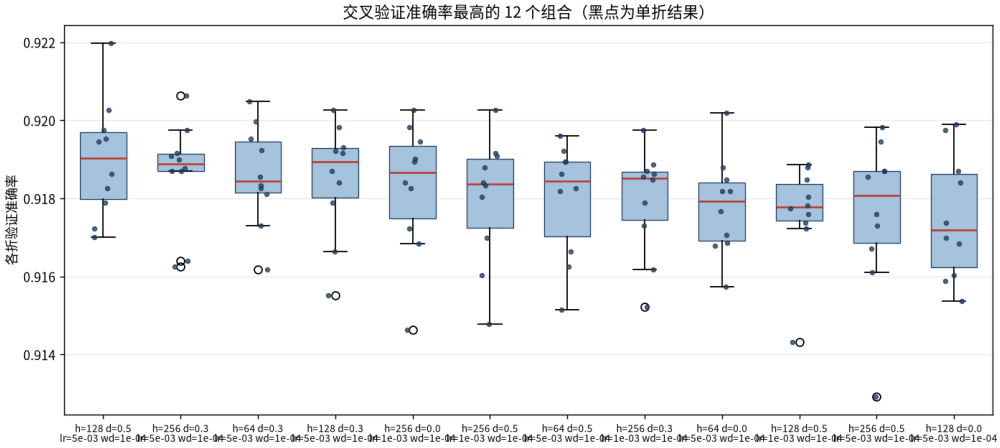


图 4-2　前 12 名组合的 10 折准确率分布

排名	hidden	dropout	lr	weight_decay	平均准确率	F1(macro)
1	128	0.5	5e-3	1e-4	**0.9190**	0.9178
2	256	0.3	5e-3	1e-4	0.9186	0.9175
3	64	0.3	5e-3	1e-4	0.9186	0.9174
34	64	0.0	5e-3	0	0.9075	0.9061
35	256	0.3	5e-3	0	0.9027	0.9012
36	256	0.0	5e-3	0	0.8984	0.8969

表 4-1　交叉验证排名前三名与后三名（完整 36 行见 6 节程序输出）

最佳组合。`hidden=128`、`dropout=0.5`、`lr=5e-3`、`weight_decay=1e-4`，交叉验证平均准确率 **0.9190 ± 0.0014**，宏平均 F1 **0.9178 ± 0.0015**。10 折准确率依次为 0.9195、0.9195、0.9172、0.9170、0.9203、0.9220、0.9179、0.9183、0.9197、0.9186，极差仅 0.0050，说明该组合在不同划分下都稳定。

最差组合。`hidden=256`、`dropout=0.0`、`lr=5e-3`、`weight_decay=0`，平均准确率 0.8984。最佳组合相对最差组合提升 **2.06 个百分点**。

最终重训与测试。以各折最优轮次的中位数 17 作为固定轮数，在完整 67349 句训练集上重训（耗时 2.0 s，训练损失由 0.4760 降到 0.1257），再在 872 句测试集上评估，结果见表 4-2 与图 4-3。

类别	Precision	Recall	F1	样本数
negative	0.8541	0.7383	0.7920	428
positive	0.7769	0.8784	0.8245	444
宏平均	0.8155	0.8083	0.8083	872
准确率	—	—	**0.8096**	872

表 4-2　最佳组合在 872 句测试集上的分类结果

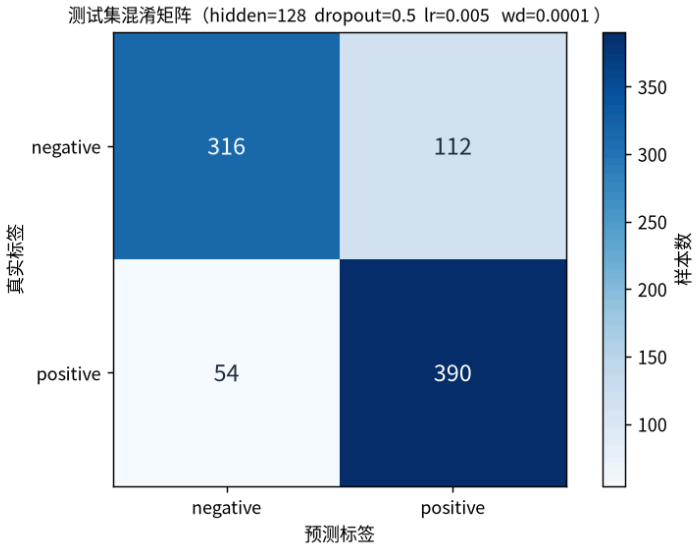


图 4-3　测试集混淆矩阵

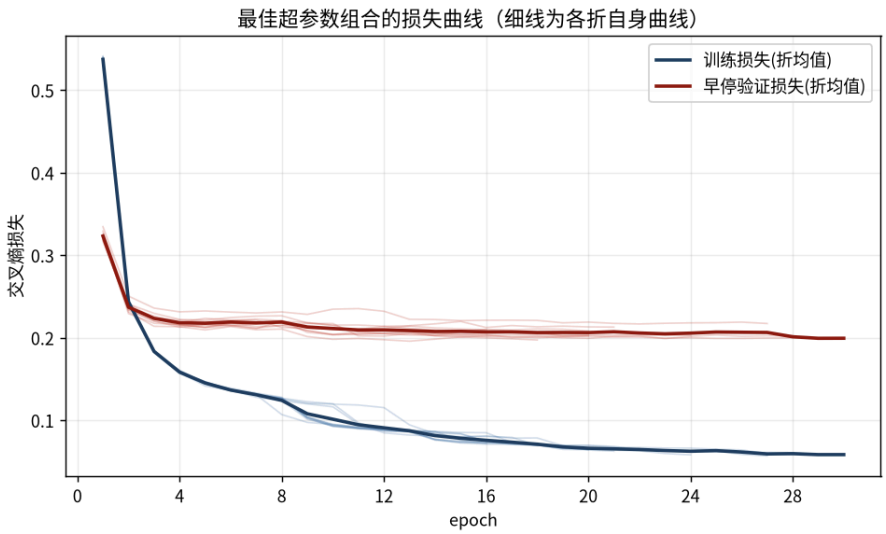


图 4-4　最佳组合在折内训练时的训练/早停验证损失曲线

测试集准确率 0.8096。两类之间存在明显偏向：`negative` 的召回只有 0.7383，而 `positive` 的召回高达 0.8784，即模型倾向于把句子判成正类。测试集本身两类基本均衡（428 / 444），因此这一偏向来自训练集——训练集中正例占 55.8%（37569 / 67349），模型学到了这个先验。

4.5 结果分析

4.5.1 四个超参数的影响极不均衡

把每个超参数在各取值上的平均准确率取出来（对其余超参数做边际平均），得到表 4-3。图 4-5 给出 `hidden_size` 与 `dropout` 的二维热力图，图 4-6 给出 `lr` 与 `weight_decay` 的边际柱状图。

超参数	取值	边际平均准确率	边际极差	相对影响
`weight_decay`	0.0	0.9100	**0.0078**	**5.2×**
	1e-4	**0.9178**		
`lr`	1e-3	0.9150	0.0023	1.5×
	5e-3	0.9128		
`hidden_size`	64	0.9144	0.0015	1.0×
	128	0.9144		
	256	0.9129		
`dropout`	0.0	0.9132	0.0013	0.9×
	0.3	0.9140		
	0.5	0.9145		

表 4-3　四个超参数的边际影响（相对影响以极差最小的 dropout 为 1.0）

结论有三点。其一，`weight_decay` 的边际极差 0.0078，是其余三个超参数的 3–6 倍，是唯一真正起作用的超参数。其二，`hidden_size` 与 `dropout` 的边际极差（0.0015、0.0013）与各折标准差（约 0.0014）同量级，即它们的影响淹没在划分噪声里——在本任务上几乎无关紧要。其三，从排名结构也能看出 `weight_decay` 的主导地位：前 18 名全部是 `weight_decay=1e-4`，后 18 名全部是 `weight_decay=0`，没有任何一个不加权重衰减的组合能挤进前半区。

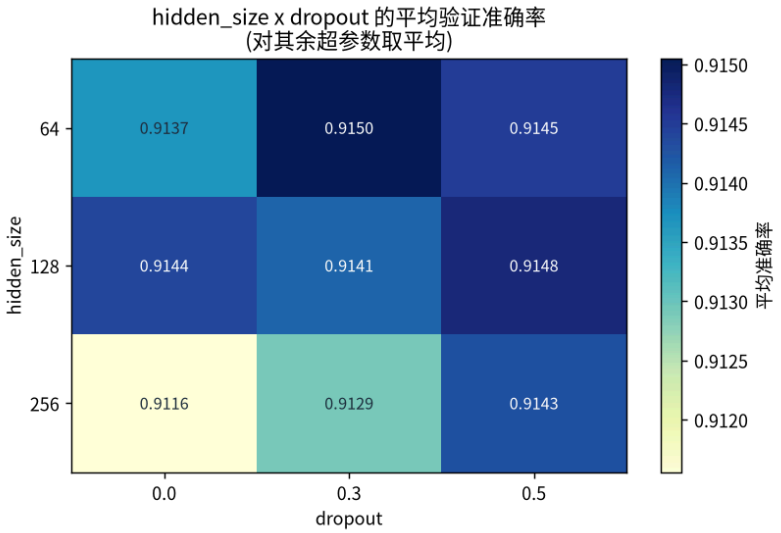


图 4-5　`hidden_size` × `dropout` 的平均验证准确率热力图

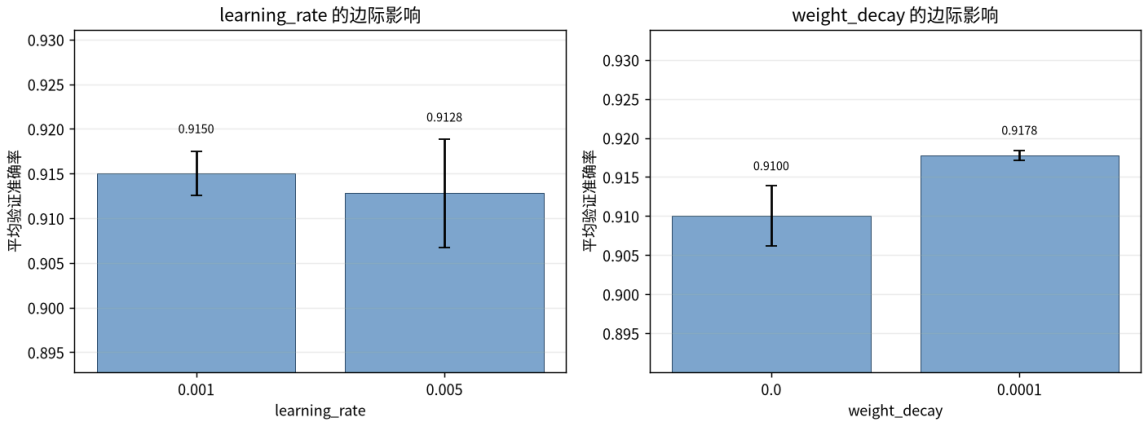


图 4-6　`lr` 与 `weight_decay` 的边际影响（误差棒为各取值的波动范围）

热力图上唯一的规律是：`hidden_size=256` 时 `dropout` 有单调正效应（0.9116 → 0.9129 → 0.9143），因为大隐层容量更大、更易过拟合，增大丢弃率能缓解；而 `hidden_size=64` 与 128 时 `dropout` 基本没有影响。整个热力图的取值区间只有 0.9116–0.9150（0.34 个百分点），进一步印证这两个超参数在本任务上不敏感。原因不难理解：输入已是 50000 维稀疏 TF-IDF，特征本身判别力很强，任务难度低，模型容量不是瓶颈。

4.5.2 单变量边际分析会误导：`lr` 与 `weight_decay` 的交互

表 4-3 显示 `lr` 的边际平均是 1e-3（0.9150）优于 5e-3（0.9128）。若只看这一个数字，会得出“学习率取 1e-3 更好”的结论——但这个结论是错的，最佳组合用的恰恰是 5e-3。

把 `lr` 与 `weight_decay` 交叉列表（表 4-4）就能看清原因：`lr` 的边际差异完全由 `weight_decay=0` 那一列造成。

`lr`	`weight_decay=0`	`weight_decay=1e-4`	组内极差
1e-3	0.9126	0.9175	0.0008 / 0.0015
5e-3	0.9075	**0.9181**	**0.0130** / 0.0017

表 4-4　`lr` × `weight_decay` 的平均验证准确率及组内波动

在 `weight_decay=1e-4` 内部，`lr=5e-3`（0.9181）反而略优于 `lr=1e-3`（0.9175），且该格的 9 个组合全部落在 0.9173–0.9190 这个很窄的区间里。而在 `weight_decay=0` 内部，`lr=5e-3` 的 9 个组合散布在 0.8984–0.9114（极差 1.30 个百分点），且普遍在第 1–3 轮就达到最优，即一上来就发散。也就是说，**不加权重衰减时高学习率会把训练推向不稳定，权重衰减把它救了回来，并让它成为表现最好的一格**。这正是网格搜索相对“固定其他超参数、逐个调一个”的价值：只有同时枚举两个超参数才能发现这种交互。

最优轮次同样印证了这一机制（表 4-5）。

lr	weight_decay=0	weight_decay=1e-4
1e-3	8.7	28.6
5e-3	2.1	18.3

表 4-5　各组内平均最优轮次（早停选出的轮次）

不加权重衰减的组合平均在第 2–9 轮就达到验证最优然后开始变差，是典型的快速过拟合；加了权重衰减的组合能一路训到第 18–29 轮仍在改善，权重衰减在这里起的是抑制过拟合、延长有效训练时间的作用。

需要说明一处局限：`weight_decay=1e-4` 且 `lr=1e-3` 的平均最优轮次为 28.6，已逼近 30 轮的上限，说明这一组组合是被轮数上限截断的，若把上限提高应还有小幅增益。由于最优组合（`lr=5e-3`）的平均最优轮次为 18.3，未受上限影响，因此这一截断不改变最终的选优结论。

4.5.3 交叉验证分数与测试集分数的落差

最佳组合的交叉验证分数 0.9190 与测试集分数 0.8096 相差 **10.94 个百分点**。这个落差不是流程缺陷，而是 SST-2 训练/测试集本身分布差异造成的：训练集句子平均 9.4 个词（最长 52），测试集平均 19.5 个词（最长 47），**测试集句子长度约为训练集的两倍**。句长差异意味着测试集里大量二元词组组合在训练折的词表中根本不存在，而 TF-IDF 的词表只在本折训练集上拟合。

为了验证这一点，本实验用 `ablation_features.py` 做了特征表示消融对照，把“TF-IDF 先经 SVD 降维、再喂稠密模型”这条常见路线与本实验的“TF-IDF 稀疏直接接入”路线放在同一套划分下对比，结果见表 4-6。

特征表示与模型	CV 准确率	测试准确率	落差
TF-IDF 稀疏 + MLP（本实验）	0.9190	0.8096	**+0.1094**
TF-IDF 稀疏 + LR	0.9037	0.8119	+0.0918
TF-IDF + SVD800 + LR	0.7981	0.7844	+0.0137
TF-IDF + SVD200 + LR	0.7255	0.7454	−0.0199

表 4-6　特征表示消融对照（同一套 5 折 × 2 次重复划分与同一测试集）

由表 4-6 可以读出三点。

第一，落差大小与模型对训练分布的拟合程度成正比，落差小并不代表模型好。两条稀疏路线（MLP 落差 10.94 点、LR 落差 9.18 点）都严重高估；两条 SVD 路线落差很小甚至为负，但那不是因为它们更稳健，而是因为它们被降维瓶颈压得几乎没学到训练分布的信息，属于严重欠拟合——其绝对分数（0.7454、0.7844）远低于稀疏路线的 0.81 左右。所以“CV 与测试集很接近”只有在模型本身已经训好的前提下才是好事。

第二，SVD 降维在本任务上损失巨大，这是本实验改用稀疏输入的依据。 SVD200 累计只解释原始稀疏特征 12.0% 的方差，测试准确率从稀疏 LR 的 0.8119 掉到 0.7454；SVD800

解释 26.4%，测试 0.7844，仍明显更差。CV 上的差距更大（0.9037 / 0.9190 对 0.7255 / 0.7981）。原因是截断 SVD 保的是方差大的方向，而情感分类最依赖的恰恰是方差不大但判别力强的稀有词与否定搭配等二元词组，降维首先丢掉的正是这些成分。

第三，交叉验证能可靠区分差距大的方案，但分辨不了 1 个百分点以内的差别。CV 把两条稀疏路线正确排在两条 SVD 路线之前（0.9037 / 0.9190 对 0.7255 / 0.7981），排序与测试集一致；但在稀疏 LR 与稀疏 MLP 之间排反了——CV 认为 MLP（0.9190）强于 LR（0.9037），测试集却是 LR（0.8119）略强于 MLP（0.8096）。差距 1.5 个百分点时 CV 的判断就不可靠了，此时应看独立测试集。这也说明本实验保留独立测试集这一步是必要的：若只信 CV，会得出“MLP 明显优于 LR”的结论。

4.6 实现级效率：稀疏张量格式与批量大小的选择

这一节记录一组实现层面的实测工作。它不改变模型或搜索空间，但决定了整个网格搜索能否在可接受的时间内跑完，也解释了并行搜索为什么加速有限。

问题现象。 最初的实现（每批构造 COO 稀疏张量、batch size 256、`hidden_size=256`）单轮训练耗时 1425 ms；按平均每组合 10 折 × 20 轮估算，36 个组合的完整网格需要数小时。同时观察到：GPU 利用率 100%，但功耗只有约 70 W（该卡上限约 115 W），且 4 进程并行的加速比接近 1.0，即进程级并行几乎没有收益。这两条线索都指向“GPU 忙在别处，不在算力上”。

定位。用 `efficiency_bench.py` 把单次迭代拆成三段计时（`hidden_size=256`、batch 2048），结果见表 4-7。

单次迭代内的环节	耗时	在完整循环中的占比
scipy 行切片（纯 CPU，不碰设备）	0.093 ms	—（是下一行的子步骤）
取一批数据（切片 + 转 CSR + 搬到设备）	0.472 ms	7.3%
模型步（前向 + 反向 + 参数更新）	**6.371 ms**	**约 93%**
完整训练循环（取数据 + 模型步）	6.445 ms	100%

表 4-7　单次迭代耗时构成（`hidden_size=256`，batch=2048）

取一批数据只占完整循环的 7.3%，耗时几乎全在模型步里。表中前三行是层层包含的关系（行切片是取数据的子步骤，取数据又是完整循环的一部分），因此不能相加；两段分开测得的耗时之和为 6.843 ms，比完整循环的 6.445 ms 略大，差额来自测量粒度。继续往下拆：单独测 `Adam.step()`这一步就要 1.6–4.0 ms（随第一层参数量增长），折合等效内存带宽 47–113 GB/s，即这一步是被显存带宽限制的，而不是被算力限制的。

机制。 第一层权重是 50000 × `hidden_size` 的稠密矩阵，`hidden_size=256` 时为 12.8M 个参数。每更新一次参数，Adam 需要读写参数、梯度、一阶动量和二阶动量，合计约 7 遍 × 12.8M × 4 B ≈ 358 MB。关键在于**这个开销只与第一层参数量有关，与批次大小无关**：一个批次喂 256 条还是 4096 条，这一步的代价完全一样。于是它成了一个按“次”计的固定开销，批次越小、一轮里的迭代次数越多，这份开销就被重复得越多次。

对策与效果。 针对上述机制做了两点改动：一是把每批的张量格式由 COO 改为 CSR，并且改为在 CPU 侧构造好再整体搬到设备（设备端做 `to_sparse_csr` 需要额外的重排，实测多花 3 倍时间）；二是把 batch size 从 256 提到 2048，直接减少迭代次数。表 4-8 是 `efficiency_bench.py` 在 `hidden_size=256` 下的实测。

输入格式	批量	每轮迭代数	单轮耗时	单次迭代
COO	256	189	1425.0 ms	7.54 ms
CSR	256	189	1100.4 ms	5.82 ms
稠密	256	189	2356.7 ms	12.47 ms
COO	2048	23	198.8 ms	8.64 ms
CSR	2048	23	**149.2 ms**	6.49 ms
稠密	2048	23	2133.1 ms	92.74 ms
CSR	4096	11	76.3 ms	6.93 ms

表 4-8　输入格式与批量大小对单轮训练耗时的影响（`hidden_size=256`）

两个观察很关键。其一，两种稀疏格式的单次迭代耗时几乎不随批量变化（COO 7.54 → 8.64 ms，CSR 5.82 → 6.93 ms），而稠密格式的耗时随批量线性增长（12.47 → 92.74 ms）并且整体慢一个数量级。这直接印证了“固定开销主导”的判断——稀疏格式受固定开销支配，所以摊薄到更多样本上更划算；稠密格式受实际算力支配，批量越大单次迭代越贵。其二，同一批量下 CSR 比 COO 快约 1.3 倍，配合批量 256 → 4096，单轮耗时从 1425.0 ms 降到 76.3 ms，合计 18.7 倍。

放大批量是否会掉准确率？必须验证，因为批量直接改变优化动态。`efficiency_bench.py` 第三节在折 0 上固定 `hidden_size=256`、`dropout=0`、`weight_decay=0` 做对比，结果见表 4-9。

批量	`lr`	最优轮	验证准确率	每折耗时	相对
256	1e-3	2	0.9115	8.4 s	1.00×
512	1e-3	2	0.9099	4.2 s	2.00×
1024	1e-3	3	0.9114	2.5 s	3.35×
2048	1e-3	5	0.9112	1.7 s	5.07×
2048	5e-3	1	0.8963	1.0 s	8.49×

表 4-9　批量大小敏感性：验证准确率与每折训练耗时（折 0，`hidden_size=256`，`dropout=0`，`weight_decay=0`）

批量由 256 放大到 2048，验证准确率从 0.9115 变到 0.9112，相差 0.0003，可以认为没有损失；耗时降到约 1/5。最终取 2048 而未取 4096，是因为 2048 时每轮仍有 23 次参数更新（30 轮共约 690 次），而 4096 会把每轮更新压到 11 次；同时 149.2 → 76.3 ms 的收益已明显递减，不值得为此进一步牺牲优化步数。另外表 4-9 最后一行也复现了 4.5.2 的结论：`weight_decay=0` 配上 `lr=5e-3` 时最优轮次是 1，即一上来就发散。

串行与并行的耗时对比。修正后重新测量，结果见表 4-10。

支	墙钟耗时	累计训练耗时	并发度	相对串行
串行 n_jobs=1	35.03 s	34.0 s	0.97	1.00×
并行 n_jobs=4	30.03 s	92.6 s	3.08	**1.17**×

表 4-10　8 个组合 × 5 折的串行/并行耗时对比（两支均固定 `torch` 线程数 = 2）

4 个进程确实做到了并发：累计训练耗时由 34.0 s 涨到 92.6 s，并发度 3.08，说明 4 个任务在同时推进；但墙钟只缩短 17%，加速比 1.17×。原因正是上面定位到的固定开销——每次参数更新都要读写 12.8M 个参数的梯度与优化器状态，这部分吃的是显存带宽，4 个进程在同一个 GPU 上排队等同一份带宽，于是单个任务被拉长约 2.7 倍（92.6 / 34.0），并发带来的收益几乎被抵消。相对地，360 次训练的完整网格那一步并发度为 3.81（602.22 s 墙钟

完成 2296.7 s 累计训练），因为各组合折数多、任务粒度更均衡，排队现象比这个 8 组合的短基准轻一些，实际加速比也更好。

结论。并行搜索对计算密集的任务有效，对本实验这种“第一层参数量大、单步计算量小、受显存带宽制约”的任务收益有限。要让并行真正起效，应当先降低每次参数更新的固定开销（例如换用对稀疏大矩阵更友好的优化器，或压缩第一层宽度），而不是单纯增加进程数。

5 实验总体结论

1. **网格搜索配合重复分层 K 折确实找到了显著更优的配置**。最佳组合 `hidden=128 / dropout=0.5 / lr=5e-3 / weight_decay=1e-4` 的交叉验证平均准确率为 0.9190 ± 0.0014，相对最差组合的 0.8984 提升 2.06 个百分点；10 折极差仅 0.0050。
2. **四个超参数的影响极不均衡**。`weight_decay` 的边际极差 0.0078，是其余三者（0.0013–0.0023）的 3–6 倍，且排名前 18 名与后 18 名恰好被 `weight_decay=1e-4` 与 `weight_decay=0` 完全分开。`hidden_size` 与 `dropout` 在本任务上的影响与折间噪声同量级，属于不敏感超参数。
3. **单变量边际分析会误导，必须看交互**。`lr` 的边际平均看似 1e-3 更优，实际完全由 `weight_decay=0` 一列拖累造成；在 `weight_decay=1e-4` 内 `lr=5e-3` 反而更好，并且正是最终的最佳组合。只看边际值会选错。这是网格搜索相对“逐个调一个超参数”的核心价值。
4. **交叉验证分数不能直接当作测试集预期**。最佳组合 CV 0.9190、测试 0.8096，落差 10.94 个百分点。消融对照表明落差源于 SST-2 训练/测试集的句长分布差异（9.4 词对 19.5 词），且落差大小与模型对训练分布的拟合程度成正比——SVD 路线的落差接近零只是因为它严重欠拟合，并非更稳健。
5. **选对特征表示比调超参数更划算**。把“TF-IDF 先经 SVD 降维、再喂稠密模型”改为“TF-IDF 稀疏直接接入”，测试准确率从 0.7844（SVD800 + LR）提升到 0.8096–0.8119，CV 上从 0.7981 提升到 0.9037–0.9190。调超参数在本任务上带来约 2 个百分点，而选对特征表示带来 3–9 个百分点。
6. **本任务的耗时主要来自实现层面的固定开销，而非算力**。单次迭代中取一批数据只占 7.3%，其余几乎全花在第一层 50000 × `hidden_size` 的参数更新上，且这一开销与批次大小无关（受显存带宽限制）。改用 CSR 并把批量提到 2048 后，单轮训练从 1425.0 ms 降到 149.2 ms；但也正因如此，4 进程并行的加速比只有 1.17×——显存带宽是串行与并行共同的瓶颈。
7. **测试集上存在类别偏向**。`positive` 的召回 0.8784 明显高于 `negative` 的 0.7383，模型倾向判为正类。测试集两类基本均衡，因此该偏向来自训练集正例占 55.8% 的先验，可通过类别加权或重采样缓解。
8. **局限与可改进之处：**
   - 早停上限 30 轮对 `weight_decay=1e-4 / lr=1e-3` 这一组已构成截断（平均最优轮次 28.6），该组分数被略微低估；提高上限可能还有小幅增益。
   - `hidden_size` 与 `dropout` 的取值在本任务上不敏感，把搜索预算分给它们收益很低；更值得搜索的是 TF-IDF 本身的参数（如 `min_df`、`ngram_range` 上限、`max_features`）或第一层的稀疏性正则。
   - 并行收益受单卡显存带宽限制，若要显著缩短网格搜索时间，需要换用多卡数据并行，或先把每次更新的固定开销降下来。

6 完整实验代码与程序输出

6.1 主程序：超参数网格搜索与重复分层 K 折交叉验证

入口脚本只负责编排流程与输出，具体实现分在四个模块里；两个辅助脚本（6.3、6.4 节）也复用这四个模块。

6.1.1 入口脚本 hyperparameter_search.py

```
"""SST-2 情感二分类：MLP 超参数网格搜索 + 重复 K 折交叉验证。

搜索空间 hidden_size [64,128,256] x dropout [0,0.3,0.5] x lr [1e-3,5e-3]
         x weight_decay [0,1e-4]，共 36 组
特征工程 TF-IDF (1-2 gram) 稀疏特征，词表与 IDF 在每折训练集内部拟合
训练策略 Adam + ReduceLROnPlateau + 早停（折内再切一份验证集专供早停）
评估指标 Accuracy / Precision / Recall / F1 (macro) / 混淆矩阵

本文件只负责编排流程与输出，实现分在 hs_core / hs_data / hs_model / hs_viz
四个模块里；两个辅助脚本 efficiency_bench.py、ablation_features.py 也复用它们。
"""

import argparse
import json
import os
import platform
import sys
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    precision_recall_fscore_support,
)

from hs_core import (
    ROOT,
    CLASS_NAMES,
    banner,
    format_combo,
    pick_device,
```

```
    save_cv_csv,
    sub_banner,
    summarize,
    to_per_fold_rows,
)
from hs_data import build_feature_matrices, build_folds, describe_dataset, load_sst2
from hs_model import MLP, build_grid, expand_grid, parallel_search, sparse_batch
from hs_viz import (
    plot_boxplot,
    plot_confusion,
    plot_heatmap,
    plot_loss_curves,
    plot_marginal_effects,
    plot_ranked_bar,
    setup_plot_style,
)


# ---------------------------------------------------------------------------
# 最终模型：全量重训与测试集评估
# ---------------------------------------------------------------------------
def refit_and_test(texts, labels, test_texts, test_labels, best, epochs, cfg):
    """用最佳组合在完整训练集上重训，并在测试集上评估。

    轮数固定为交叉验证中各折最优轮的平均值，不再早停：训练集已全部用上，
    没有多余数据可留作早停集，硬切一份出来只会白白少训样本。
    """
    sub_banner(f"最终重训：{format_combo(best)}，固定 epoch={epochs}，训练集全量 {len(texts)} 句")
    torch.set_num_threads(max(1, (os.cpu_count() or 4) // 2))
    X_train, X_test, vec = build_feature_matrices(texts, test_texts, cfg)
    print(f"  特征维度={X_train.shape[1]} 词表={len(vec.vocabulary_)} "
          f"非零特征/句={X_train.nnz / X_train.shape[0]:.1f}")

    device = pick_device(cfg["device"])
    torch.manual_seed(cfg["seed"])
    model = MLP(X_train.shape[1], best["hidden_size"], dropout=best["dropout"]).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=best["lr"],
                                 weight_decay=best["weight_decay"])
    criterion = nn.CrossEntropyLoss()
```

```
    y = torch.from_numpy(labels).to(device)
    n = X_train.shape[0]
    history = []
    t0 = time.time()
    for epoch in range(1, epochs + 1):
        model.train()
        perm = np.random.default_rng(cfg["seed"] + epoch).permutation(n)
        total = 0.0
        for start in range(0, n, cfg["batch_size"]):
            idx = perm[start:start + cfg["batch_size"]]
            optimizer.zero_grad()
            logits = model(sparse_batch(X_train, idx, device))
            loss = criterion(logits, y[torch.from_numpy(idx).to(device)])
            loss.backward()
            optimizer.step()
            total += loss.item() * len(idx)
        history.append(total / n)
        print(f"    epoch {epoch:3d}/{epochs}  训练损失={history[-1]:.4f}")

    model.eval()
    with torch.no_grad():
        logits = model(sparse_batch(X_test, np.arange(X_test.shape[0]), device))
    preds = logits.argmax(dim=1).cpu().numpy()
    print(f"  重训耗时 {time.time() - t0:.1f}s")
    return model, preds, history


def print_test_report(test_labels, preds):
    """打印测试集指标。宏平均对两个类别同等加权，避免被多数类带走。"""
    acc = accuracy_score(test_labels, preds)
    p, r, f1, _ = precision_recall_fscore_support(
        test_labels, preds, average="macro", zero_division=0
    )
    print(f"\n  测试集准确率 : {acc:.4f}")
    print(f"  宏平均 Precision={p:.4f}  Recall={r:.4f}  F1={f1:.4f}")
    print()
    print(classification_report(test_labels, preds, target_names=CLASS_NAMES,
                                digits=4, zero_division=0))
    return {"accuracy": float(acc), "precision_macro": float(p),
            "recall_macro": float(r), "f1_macro": float(f1)}
```

```python
# ---------------------------------------------------------------------------
# 命令行
# ---------------------------------------------------------------------------
def parse_args():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--data-dir", type=Path, default=ROOT / "demo_labs" / "SST-2")
    p.add_argument("--out-dir", type=Path, default=Path(__file__).resolve().parent)
    p.add_argument("--n-splits", type=int, default=5)
    p.add_argument("--n-repeats", type=int, default=2)
    p.add_argument("--epochs", type=int, default=30)
    p.add_argument("--patience", type=int, default=5)
    p.add_argument("--batch-size", type=int, default=2048)
    p.add_argument("--max-features", type=int, default=50000)
    p.add_argument("--min-df", type=int, default=3)
    p.add_argument("--jobs", type=int, default=4)
    p.add_argument("--torch-threads", type=int, default=2,
                   help="每个训练进程固定的 CPU 线程数，串行与并行两支一致")
    p.add_argument("--bench-combos", type=int, default=8,
                   help="串行/并行耗时对比所用的组合数，设为 0 则跳过对比")
    p.add_argument("--device", default="auto")
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--quick", action="store_true", help="小网格冒烟测试")
    return p.parse_args()


# ---------------------------------------------------------------------------
# 主流程
# ---------------------------------------------------------------------------
def main():
    args = parse_args()
    cfg = {
        "n_splits": args.n_splits,
        "n_repeats": args.n_repeats,
        "epochs": args.epochs,
        "patience": args.patience,
        "batch_size": args.batch_size,
        "seed": args.seed,
        "device": args.device,
```

```
    "torch_threads": args.torch_threads,
    "tfidf": {"ngram_range": (1, 2), "min_df": args.min_df,
              "max_features": args.max_features, "sublinear_tf": True},
}
figures_dir = args.out_dir / "figures"
results_dir = args.out_dir / "results"
figures_dir.mkdir(parents=True, exist_ok=True)
results_dir.mkdir(parents=True, exist_ok=True)

program_start = time.time()
banner("SST-2 情感二分类：MLP 超参数网格搜索 + 重复 K 折交叉验证")
print(f"  Python        : {sys.version.split()[0]} ({platform.system()})")
print(f"  PyTorch       : {torch.__version__}")
print(f"  计算设备      : {pick_device(args.device)}"
      + (f" ({torch.cuda.get_device_name(0)})" if torch.cuda.is_available()
         and pick_device(args.device).type == "cuda" else ""))
print(f"  随机种子      : {args.seed}")
print(f"  折数设置      : {args.n_splits} 折 x {args.n_repeats} 次重复 = "
      f"{args.n_splits * args.n_repeats} 个训练/验证划分")

banner("数据集", char="-")
texts, labels = load_sst2(args.data_dir / "train.tsv")
test_texts, test_labels = load_sst2(args.data_dir / "test.tsv")
describe_dataset("训练集", labels, np.array([len(t.split()) for t in texts]))
describe_dataset("测试集", test_labels, np.array([len(t.split()) for t in test_texts]))

combos = expand_grid(build_grid(args.quick))
banner("超参数搜索空间", char="-")
for key, values in build_grid(args.quick).items():
    print(f"  {key:<14}: {values}")
total_fits = len(combos) * args.n_splits * args.n_repeats
print(f"  组合数        : {len(combos)}")
print(f"  预计训练次数  : {len(combos)} x {args.n_splits * args.n_repeats} "
      f"= {total_fits}")

banner("构建交叉验证划分与折内特征", char="-")
print("  说明：TF-IDF 的词表与 IDF 只在本折训练集上拟合，再变换验证折；")
print("        早停使用折内再切出的 10% 验证集，与外层评估折严格分离。")
t0 = time.time()
folds = build_folds(texts, labels, cfg)
```

```
print(f"  划分与特征构建总耗时 {time.time() - t0:.2f}s, "
      f"外层验证样本合计 {sum(len(f['y_val']) for f in folds)}")

timing = None
if args.bench_combos > 1:
    bench = combos[:args.bench_combos]
    bench_folds = [f for f in folds if f["repeat"] == 0]
    banner("串行与并行搜索耗时对比", char="-")
    print(f"  在 {len(bench)} 个组合 x {len(bench_folds)} 折 上分别执行串行与并行搜索")
    print(f"  两支均固定 torch 线程数 = {args.torch_threads}, "
          f"因此加速比只反映进程级并行")
    _, serial_wall = parallel_search(bench, bench_folds, cfg, 1, "串行搜索")
    _, par_wall = parallel_search(bench, bench_folds, cfg, args.jobs, "并行搜索")
    timing = {"combos": len(bench), "folds": len(bench_folds),
              "serial_seconds": serial_wall, "parallel_seconds": par_wall,
              "jobs": args.jobs, "speedup": serial_wall / par_wall}
    print(f"  >> 加速比 {timing['speedup']:.2f}x "
          f"(串行 {serial_wall:.2f}s -> 并行 {par_wall:.2f}s, n_jobs={args.jobs})")

banner("网格搜索（完整搜索空间，并行执行）", char="-")
records, search_wall = parallel_search(combos, folds, cfg, args.jobs, "完整网格搜索")

summaries = [summarize(r) for r in records]
summaries.sort(key=lambda r: r["mean_accuracy"], reverse=True)
by_key = {tuple(sorted(r["combo"].items())): r for r in records}
best_summary = summaries[0]
best_combo = {k: best_summary[k] for k in ("hidden_size", "dropout", "lr", "weight_decay")}
best_record = by_key[tuple(sorted(best_combo.items()))]

banner("交叉验证结果排名（按平均准确率降序）", char="-")
print(f"  {'#':>3}  {'hidden':>7} {'dropout':>8} {'lr':>8} {'weight_decay':>13} "
      f"{'准确率':>10} {'标准差':>9} {'F1(macro)':>10} {'最优epoch':>9}")
print(f"  {'-' * 84}")
for i, row in enumerate(summaries, 1):
    print(f"  {i:>3}  {row['hidden_size']:>7} {row['dropout']:>8.1f} "
          f"{row['lr']:>8.0e} {row['weight_decay']:>13.0e} "
          f"{row['mean_accuracy']:>10.4f} {row['std_accuracy']:>9.4f} "
          f"{row['mean_f1']:>10.4f} {row['mean_best_epoch']:>9.1f}")

banner("最佳超参数组合", char="-")
```

```
print(f"  {format_combo(best_combo)}")
print(f"  交叉验证平均准确率 : {best_summary['mean_accuracy']:.4f} "
      f"± {best_summary['std_accuracy']:.4f}")
print(f"  交叉验证宏平均 F1  : {best_summary['mean_f1']:.4f} "
      f"± {best_summary['std_f1']:.4f}")
fold_accs = "  ".join(f"{f['accuracy']:.4f}" for f in best_record["folds"])
print(f"  各折准确率          : [{fold_accs}]")

worst = summaries[-1]
print(f"\n  对照：最差组合 {format_combo({k: worst[k] for k in best_combo})}")
print(f"        交叉验证平均准确率 {worst['mean_accuracy']:.4f}")
print(f"        最佳组合相对最差组合提升 "
      f"{(best_summary['mean_accuracy'] - worst['mean_accuracy']) * 100:.2f} 个百分点")

epochs = max(5, int(round(best_summary["mean_best_epoch"])))
model, test_preds, _ = refit_and_test(texts, labels, test_texts, test_labels,
                                      best_combo, epochs, cfg)

banner("测试集评估（最佳组合，完整训练集重训）", char="-")
test_metrics = print_test_report(test_labels, test_preds)

banner("可视化输出", char="-")
setup_plot_style()
plot_ranked_bar(summaries, figures_dir / "01_组合排名.png")
plot_boxplot(summaries, figures_dir / "02_top12_箱线图.png")
plot_heatmap(summaries, figures_dir / "03_hidden_dropout_热力图.png")
plot_marginal_effects(summaries, figures_dir / "04_学习率与权重衰减_边际影响.png")
plot_loss_curves(best_record, figures_dir / "05_最佳组合损失曲线.png")
cm = plot_confusion(test_labels, test_preds, figures_dir / "06_测试集混淆矩阵.png",
                    f"测试集混淆矩阵（{format_combo(best_combo)}）")
for path in sorted(figures_dir.glob("*.png")):
    print(f"  已保存 {path.relative_to(args.out_dir)}")

banner("结果落盘", char="-")
summary_csv, fold_csv = save_cv_csv(summaries, to_per_fold_rows(records),
                                    results_dir / "cv_summary.csv")
torch.save(model.state_dict(), results_dir / "best_model.pt")
payload = {
    "best_combo": {k: (float(v) if isinstance(v, float) else v)
                   for k, v in best_combo.items()},
```

```
        "cv_mean_accuracy": best_summary["mean_accuracy"],
        "cv_std_accuracy": best_summary["std_accuracy"],
        "cv_mean_f1_macro": best_summary["mean_f1"],
        "test_metrics": test_metrics,
        "confusion_matrix": cm.tolist(),
        "final_epochs": epochs,
        "timing": timing,
        "n_splits": args.n_splits,
        "n_repeats": args.n_repeats,
        "grid": build_grid(args.quick),
        "total_fits": total_fits,
    }
    with open(results_dir / "best_params.json", "w", encoding="utf-8") as fh:
        json.dump(payload, fh, ensure_ascii=False, indent=2)

    for path in (summary_csv, fold_csv, results_dir / "best_model.pt",
                 results_dir / "best_params.json"):
        print(f"  已保存 {path.relative_to(args.out_dir)}")

    banner("完成")
    print(f"  完整网格搜索墙钟耗时 : {search_wall:.2f}s")
    print(f"  程序总耗时           : {time.time() - program_start:.2f}s")


if __name__ == "__main__":
    main()
```

6.1.2 公共模块 hs_core.py：输出格式、设备选择、结果落盘

```
"""公共基础设施：输出格式、设备选择、组合格式化、结果整理，不含实验逻辑。"""

import csv
import warnings
from pathlib import Path

import numpy as np
import torch

ROOT = Path(__file__).resolve().parent.parent
CLASS_NAMES = ["negative", "positive"]

# 每批次都新建稀疏张量，跳过程序化不变量检查可省去重复校验的开销
torch.sparse.check_sparse_tensor_invariants.disable()


def silence_sparse_warnings():
    """屏蔽 PyTorch 建稀疏张量的两条提示（CSR 仍是 beta、不变量检查已关闭）。

    joblib 的工作进程导入后会重置警告过滤器，所以每个组合开跑前要再设一次。
    """
    warnings.filterwarnings("ignore", message=".*Sparse.*")


silence_sparse_warnings()


# ---------------------------------------------------------------------------
# 输出格式
# ---------------------------------------------------------------------------
def banner(title, char="=", width=78):
    print()
    print(char * width)
    print(f"  {title}")
    print(char * width)


def sub_banner(title, width=78):
```

```
    print()
    print("-" * width)
    print(f"  >> {title}")
    print("-" * width)


def format_combo(combo):
    return (f"hidden={combo['hidden_size']:<4d} dropout={combo['dropout']:<4.1f} "
            f"lr={combo['lr']:<7.4g} wd={combo['weight_decay']:<7.4g}")


def pick_device(requested):
    if requested == "auto":
        return torch.device("cuda" if torch.cuda.is_available() else "cpu")
    return torch.device(requested)


# ---------------------------------------------------------------------------
# 结果整理
# ---------------------------------------------------------------------------
def summarize(record):
    accs = np.array([f["accuracy"] for f in record["folds"]])
    per_fold = {m: np.array([f[m] for f in record["folds"]])
                for m in ("precision_macro", "recall_macro", "f1_macro")}
    return {
        **record["combo"],
        "folds": record["folds"],
        "mean_accuracy": float(accs.mean()),
        "std_accuracy": float(accs.std()),
        "mean_precision": float(per_fold["precision_macro"].mean()),
        "mean_recall": float(per_fold["recall_macro"].mean()),
        "mean_f1": float(per_fold["f1_macro"].mean()),
        "std_f1": float(per_fold["f1_macro"].std()),
        "mean_best_epoch": float(np.mean([f["best_epoch"] for f in record["folds"]])),
    }


def save_cv_csv(summaries, per_fold, path):
    fields = ["hidden_size", "dropout", "lr", "weight_decay", "mean_accuracy",
              "std_accuracy", "mean_precision", "mean_recall", "mean_f1", "std_f1",
```

```
            "mean_best_epoch"]
    with open(path, "w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=fields)
        writer.writeheader()
        for row in summaries:
            writer.writerow({k: row[k] for k in fields})

    fold_path = path.with_name("cv_per_fold.csv")
    with open(fold_path, "w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(["hidden_size", "dropout", "lr", "weight_decay",
                         "repeat", "fold", "accuracy", "precision_macro",
                         "recall_macro", "f1_macro", "best_epoch", "train_seconds"])
        for row in per_fold:
            writer.writerow(row)
    return path, fold_path


def to_per_fold_rows(records):
    rows = []
    for rec in records:
        for f in rec["folds"]:
            rows.append([rec["combo"]["hidden_size"], rec["combo"]["dropout"],
                         rec["combo"]["lr"], rec["combo"]["weight_decay"],
                         f["repeat"], f["fold"], f["accuracy"], f["precision_macro"],
                         f["recall_macro"], f["f1_macro"], f["best_epoch"],
                         f["train_seconds"]])
    return rows
```

6.1.3 数据模块 hs_data.py：SST-2 读取与折内 TF-IDF 特征流水线

```
"""数据读取，以及折内拟合的 TF-IDF 特征流水线。

词表与 IDF 都在每折训练集内部拟合，验证折只用训练折学到的词表做变换。
"""

import csv
import time

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import RepeatedStratifiedKFold, train_test_split

# 早停要另切一份数据，不能拿外层验证折来用：外层验证折一旦参与"什么时候停"
# 这个决策，它就不再是模型没见过的数据，交叉验证分数会被系统性抬高。
INNER_VAL_FRACTION = 0.1


def load_sst2(path):
    """读取 SST-2 的制表符分隔文件，返回 (句子列表, 标签数组)。"""
    texts, labels = [], []
    with open(path, encoding="utf-8") as fh:
        reader = csv.reader(fh, delimiter="\t")
        header = next(reader)
        if [h.strip().lower() for h in header[:2]] != ["sentence", "label"]:
            raise ValueError(f"{path} 表头不符合预期: {header}")
        for row in reader:
            if len(row) < 2 or not row[1].strip():
                continue
            texts.append(row[0].strip())
            labels.append(int(row[1]))
    return texts, np.asarray(labels, dtype=np.int64)  # text: list[str], labels: np.ndarray


def describe_dataset(name, labels, n_words):
    counts = np.bincount(labels, minlength=2)
    print(f"  {name}: {len(labels)} 句, 标签分布 neg={counts[0]} / pos={counts[1]}, "
          f"句长(词) min={n_words.min()} max={n_words.max()} 均值={n_words.mean():.1f}")
```

```
def build_feature_matrices(fit_texts, apply_texts, cfg):
    """在 fit_texts 上拟合 TF-IDF，再变换 apply_texts，返回稀疏矩阵。

    词表与 IDF 只用 fit_texts 学习，验证折的词频信息不会进入特征。
    """
    vectorizer = TfidfVectorizer(**cfg["tfidf"])
    csr_fit = vectorizer.fit_transform(fit_texts).astype(np.float32)
    csr_apply = vectorizer.transform(apply_texts).astype(np.float32)
    return csr_fit, csr_apply, vectorizer


def build_folds(texts, labels, cfg):
    """按重复分层 K 折切分，并为每折预生成特征矩阵。

    分层保证每折正负比例与整体一致。重复多次换随机划分，是为了看清"组合
    之间的差距"有没有超过"划分本身的波动"——两者量级相当时排名就不能当真。
    特征拟合与内层切分都与超参数无关，所以每折只做一次，所有组合复用。

    以 5 折 x 2 次重复为例，每次划分产出的对象与形状：
        train_idx / val_idx        (53879,) / (13470,) int64
        fit_texts / apply_texts    list[str], 53879 / 13470 句
        y_fit / y_val              (53879,) / (13470,)
        53879 + 13470 = 67349
    """
    splitter = RepeatedStratifiedKFold(
        n_splits=cfg["n_splits"], n_repeats=cfg["n_repeats"], random_state=cfg["seed"]
    )
    folds = []
    total = cfg["n_splits"] * cfg["n_repeats"]
    for fold_id, (train_idx, val_idx) in enumerate(splitter.split(texts, labels)):
        t0 = time.time()
        fit_texts = [texts[i] for i in train_idx]
        apply_texts = [texts[i] for i in val_idx]
        y_fit = labels[train_idx]
        y_val = labels[val_idx]

        inner_idx, inner_val_idx = train_test_split(
            np.arange(len(fit_texts)),
            test_size=INNER_VAL_FRACTION,
```

```
            stratify=y_fit,
            random_state=cfg["seed"] + fold_id,
        )
        X_fit, X_val, vec = build_feature_matrices(fit_texts, apply_texts, cfg)

        folds.append(
            {
                "repeat": fold_id // cfg["n_splits"],
                "fold": fold_id % cfg["n_splits"],
                "X_inner": X_fit[inner_idx],
                "y_inner": y_fit[inner_idx],
                "X_early": X_fit[inner_val_idx],
                "y_early": y_fit[inner_val_idx],
                "X_val": X_val,
                "y_val": y_val,
                "vocab_size": len(vec.vocabulary_),
            }
        )
        print(f"    [折 {fold_id + 1:2d}/{total}] repeat={folds[-1]['repeat']} "
              f"fold={folds[-1]['fold']} 内层训练={len(inner_idx)} "
              f"早停验证={len(inner_val_idx)} 外层验证={len(val_idx)} "
              f"词表={folds[-1]['vocab_size']} "
              f"非零特征/句={X_fit.nnz / X_fit.shape[0]:.1f} "
              f"耗时={time.time() - t0:.2f}s")
    return folds
```

6.1.4 模型模块 hs_model.py：MLP、单折训练循环、网格与并行搜索

```
"""MLP 模型、单折训练循环、超参数网格与并行搜索。"""

import time
from itertools import product

import numpy as np
import torch
import torch.nn as nn
from joblib import Parallel, delayed
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

from hs_core import format_combo, pick_device, silence_sparse_warnings, sub_banner


class MLP(nn.Module):
    """单隐层多层感知机。第一层以稀疏矩阵乘法实现，输入无需稠密化。"""

    def __init__(self, input_dim, hidden_size, num_classes=2, dropout=0.0):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_size)  # 降维：(2048, 50000) -> (2048, hidden)
        self.act = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        if x.is_sparse:
            hidden = torch.sparse.mm(x, self.fc1.weight.t()) + self.fc1.bias
        else:
            hidden = self.fc1(x)
        return self.fc2(self.dropout(self.act(hidden)))


def sparse_batch(csr, idx, device):
    """把一个批次的行转成 CSR 稀疏张量并搬到设备上。

    转换放在 CPU 侧做完再整体搬运：在 GPU 上做 to_sparse_csr() 需要设备端
    重排，实测多花 3 倍时间。CUDA 上 torch.sparse.mm 的开销按"次"计而非
    按数据量计，所以稀疏格式每轮耗时几乎不随批量变化（见 4.6 节）。
```

```
    """
    sub = csr[idx]
    return torch.sparse_csr_tensor(
        torch.from_numpy(sub.indptr.astype(np.int64)),
        torch.from_numpy(sub.indices.astype(np.int64)),
        torch.from_numpy(sub.data.astype(np.float32, copy=False)),
        size=sub.shape,
    ).to(device)


def train_fold(combo, fold, cfg, device):
    """在单个折上训练一个组合，早停后在外层验证折上评估。"""
    torch.manual_seed(cfg["seed"] + fold["fold"])

    input_dim = fold["X_inner"].shape[1]
    model = MLP(input_dim, combo["hidden_size"], dropout=combo["dropout"]).to(device)
    optimizer = torch.optim.Adam(
        model.parameters(), lr=combo["lr"], weight_decay=combo["weight_decay"]
    )
    # 调度器和早停盯同一份内层验证损失。学习率减半后验证损失还能再降一段，
    # 所以早停的 patience 必须比调度器的宽，否则刚降完学习率就被停掉。
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="min", factor=0.5, patience=2, min_lr=1e-5
    )
    criterion = nn.CrossEntropyLoss()

    y_inner = torch.from_numpy(fold["y_inner"]).to(device)
    y_early = torch.from_numpy(fold["y_early"]).to(device)

    n = fold["X_inner"].shape[0]
    history = {"train_loss": [], "val_loss": []}
    best_loss, best_epoch, best_state = float("inf"), 0, None
    t0 = time.time()

    early_idx = np.arange(len(fold["y_early"]))
    for epoch in range(1, cfg["epochs"] + 1):
        model.train()
        perm = np.random.default_rng(cfg["seed"] + epoch).permutation(n)
        total = 0.0
        for start in range(0, n, cfg["batch_size"]):
```

```
            idx = perm[start:start + cfg["batch_size"]]
            optimizer.zero_grad()
            logits = model(sparse_batch(fold["X_inner"], idx, device))
            loss = criterion(logits, y_inner[torch.from_numpy(idx).to(device)])
            loss.backward()
            optimizer.step()
            # 按批内样本数加权，最后一个不满批不会把均值拉偏，
            # 损失曲线也因此与批量大小无关。
            total += loss.item() * len(idx)

        model.eval()
        with torch.no_grad():
            val_loss = criterion(
                model(sparse_batch(fold["X_early"], early_idx, device)), y_early
            ).item()
        train_loss = total / n
        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        scheduler.step(val_loss)

        if val_loss < best_loss - 1e-5:
            best_loss, best_epoch = val_loss, epoch
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
        elif epoch - best_epoch >= cfg["patience"]:
            break

    if best_state is not None:
        model.load_state_dict(best_state)

    model.eval()
    with torch.no_grad():
        logits = model(sparse_batch(fold["X_val"], np.arange(len(fold["y_val"])), device))
    preds = logits.argmax(dim=1).cpu().numpy()
    y_true = fold["y_val"]

    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true, preds, average="macro", zero_division=0
    )
    return {
        "accuracy": float(accuracy_score(y_true, preds)),
```

```
        "precision_macro": float(precision),
        "recall_macro": float(recall),
        "f1_macro": float(f1),
        "best_epoch": best_epoch,
        "train_seconds": time.time() - t0,
        "history": history,
        "repeat": fold["repeat"],
        "fold": fold["fold"],
        "preds": preds,
        "y_true": y_true,
    }


def run_combo(combo, folds, cfg):
    """在一个超参数组合上跑完整的所有折。

    每个工作进程固定使用的 CPU 线程数，使串行与并行两支的对比只反映
    进程级并行，而不会因为串行那支独占全部核心而失真。
    """
    silence_sparse_warnings()
    torch.set_num_threads(cfg["torch_threads"])
    device = pick_device(cfg["device"])
    results = [train_fold(combo, fold, cfg, device) for fold in folds]
    accs = np.array([r["accuracy"] for r in results])
    f1s = np.array([r["f1_macro"] for r in results])
    return {
        "combo": combo,
        "folds": results,
        "mean_accuracy": float(accs.mean()),
        "std_accuracy": float(accs.std()),
        "mean_f1": float(f1s.mean()),
        "std_f1": float(f1s.std()),
        "total_seconds": float(sum(r["train_seconds"] for r in results)),
    }


def parallel_search(combos, folds, cfg, n_jobs, label):
    """对一组超参数组合执行搜索，n_jobs=1 时为串行。"""
    sub_banner(f"{label}: {len(combos)} 个组合 x {len(folds)} 折 (n_jobs={n_jobs}) ")
    t0 = time.time()
```

```
    results = []
    try:
        stream = Parallel(n_jobs=n_jobs, return_as="generator_unordered")(
            delayed(run_combo)(c, folds, cfg) for c in combos
        )
    except TypeError:
        stream = iter(
            Parallel(n_jobs=n_jobs)(delayed(run_combo)(c, folds, cfg) for c in combos)
        )

    for i, res in enumerate(stream, 1):
        results.append(res)
        print(f"  [{i:2d}/{len(combos)}] {format_combo(res['combo'])}  "
              f"acc={res['mean_accuracy']:.4f} ± {res['std_accuracy']:.4f}  "
              f"f1={res['mean_f1']:.4f}  "
              f"训练耗时={res['total_seconds']:.1f}s")
    wall = time.time() - t0
    print(f"  {label} 完成，墙钟耗时 {wall:.2f}s，累计训练耗时 "
          f"{sum(r['total_seconds'] for r in results):.1f}s")
    return results, wall


# ---------------------------------------------------------------------------
# 搜索空间
# ---------------------------------------------------------------------------
def build_grid(quick=False):
    if quick:
        return {"hidden_size": [64], "dropout": [0.0, 0.3], "lr": [1e-3],
                "weight_decay": [0.0]}
    return {"hidden_size": [64, 128, 256], "dropout": [0.0, 0.3, 0.5],
            "lr": [1e-3, 5e-3], "weight_decay": [0.0, 1e-4]}


def expand_grid(grid):
    keys = list(grid)
    return [dict(zip(keys, values)) for values in product(*(grid[k] for k in keys))]
```

6.1.5 可视化模块 hs_viz.py：六张结果图

```
"""六张结果图：组合排名、Top-12 箱线图、hidden×dropout 热力图、
学习率与权重衰减的边际影响、最佳组合损失曲线、测试集混淆矩阵。"""

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
import numpy as np
from sklearn.metrics import confusion_matrix

from hs_core import CLASS_NAMES


def setup_plot_style():
    for font in ("Noto Sans CJK SC", "Noto Sans CJK JP", "Source Han Sans SC",
                 "WenQuanYi Micro Hei", "DejaVu Sans"):
        try:
            import matplotlib.font_manager as fm

            if any(font == f.name for f in fm.fontManager.ttflist):
                plt.rcParams["font.sans-serif"] = [font, "DejaVu Sans"]
                break
        except Exception:
            continue
    plt.rcParams["axes.unicode_minus"] = False
    plt.rcParams["figure.dpi"] = 130


def plot_ranked_bar(summaries, path):
    rows = sorted(summaries, key=lambda r: r["mean_accuracy"])
    fig, ax = plt.subplots(figsize=(12, 5))
    xs = np.arange(len(rows))
    ax.bar(xs, [r["mean_accuracy"] for r in rows],
           yerr=[r["std_accuracy"] for r in rows], capsize=2.5,
           color="#3b6ea5", edgecolor="#1f3f5f", linewidth=0.4)
    ax.set_xticks(xs)
    ax.set_xticklabels(
```

```
        [f"{r['hidden_size']}/{r['dropout']:.1f}/{r['lr']:.0e}/{r['weight_decay']:.0e}"
         for r in rows], rotation=90, fontsize=6)
    ax.set_xlabel("组合 (hidden/dropout/lr/weight_decay)，按平均准确率升序")
    ax.set_ylabel("交叉验证平均准确率")
    ax.set_title(f"全部 {len(rows)} 个超参数组合的交叉验证表现（误差棒为各折标准差）")
    lo, hi = min(r["mean_accuracy"] for r in rows), max(r["mean_accuracy"] for r in rows)
    ax.set_ylim(max(0, lo - 0.02), min(1.0, hi + 0.012))
    ax.axhline(max(r["mean_accuracy"] for r in rows), color="#c0392b",
               linestyle="--", linewidth=0.9)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_boxplot(summaries, path, top_n=12):
    rows = sorted(summaries, key=lambda r: r["mean_accuracy"], reverse=True)[:top_n]
    fig, ax = plt.subplots(figsize=(11, 5))
    data = [[f["accuracy"] for f in r["folds"]] for r in rows]
    labels = [f"h={r['hidden_size']} d={r['dropout']:.1f}\nlr={r['lr']:.0e} wd={r['weight_decay']:.0e}"
              for r in rows]
    bp = ax.boxplot(data, tick_labels=labels, patch_artist=True, widths=0.6)
    for box in bp["boxes"]:
        box.set(facecolor="#a8c4e0", edgecolor="#1f3f5f", linewidth=0.7)
    for med in bp["medians"]:
        med.set(color="#c0392b", linewidth=1.4)
    rng = np.random.default_rng(0)
    for i, vals in enumerate(data, 1):
        ax.scatter(np.full(len(vals), i) + rng.uniform(-0.12, 0.12, len(vals)),
                   vals, s=10, color="#1f3f5f", alpha=0.75, zorder=3)
    ax.set_ylabel("各折验证准确率")
    ax.set_title(f"交叉验证准确率最高的 {top_n} 个组合（黑点为单折结果）")
    ax.tick_params(axis="x", labelsize=7.5)
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_heatmap(summaries, path):
```

```
    hiddens = sorted({r["hidden_size"] for r in summaries})
    dropouts = sorted({r["dropout"] for r in summaries})
    grid = np.zeros((len(hiddens), len(dropouts)))
    count = np.zeros_like(grid)
    for r in summaries:
        i = hiddens.index(r["hidden_size"])
        j = dropouts.index(r["dropout"])
        grid[i, j] += r["mean_accuracy"]
        count[i, j] += 1
    grid = grid / np.maximum(count, 1)

    fig, ax = plt.subplots(figsize=(6.4, 4.4))
    im = ax.imshow(grid, cmap="YlGnBu", aspect="auto")
    ax.set_xticks(range(len(dropouts)), [f"{d:.1f}" for d in dropouts])
    ax.set_yticks(range(len(hiddens)), [str(h) for h in hiddens])
    ax.set_xlabel("dropout")
    ax.set_ylabel("hidden_size")
    ax.set_title("hidden_size x dropout 的平均验证准确率\n(对其余超参数取平均)")
    for i in range(len(hiddens)):
        for j in range(len(dropouts)):
            val = grid[i, j]
            ax.text(j, i, f"{val:.4f}", ha="center", va="center", fontsize=9,
                    color="white" if val > grid.mean() else "#20303f")
    fig.colorbar(im, ax=ax, label="平均准确率")
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_marginal_effects(summaries, path):
    """对每个超参数取值在其余超参数上取平均，画边际影响。

    边际曲线会抹平交互作用：某个取值看着更好，可能只在另一个超参数的
    某一段上成立，要配合二维交叉表读。
    """
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    for ax, key, label in zip(axes, ("lr", "weight_decay"),
                              ("learning_rate", "weight_decay")):
        values = sorted({r[key] for r in summaries})
        means = [np.mean([r["mean_accuracy"] for r in summaries if r[key] == v])
```

```
                for v in values]
        stds = [np.std([r["mean_accuracy"] for r in summaries if r[key] == v])
                for v in values]
        ax.bar([str(v) for v in values], means, yerr=stds, capsize=4,
               color="#7fa8cd", edgecolor="#1f3f5f", linewidth=0.5)
        for i, (m, s) in enumerate(zip(means, stds)):
            ax.text(i, m + s + 0.002, f"{m:.4f}", ha="center", fontsize=8)
        ax.set_xlabel(label)
        ax.set_ylabel("平均验证准确率")
        ax.set_title(f"{label} 的边际影响")
        ax.set_ylim(min(means) - 0.02, max(means) + 0.016)
        ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_loss_curves(best_record, path):
    fig, ax = plt.subplots(figsize=(7.5, 4.6))
    train_curves = [np.array(f["history"]["train_loss"]) for f in best_record["folds"]]
    val_curves = [np.array(f["history"]["val_loss"]) for f in best_record["folds"]]
    for tc, vc in zip(train_curves, val_curves):
        ax.plot(range(1, len(tc) + 1), tc, color="#3b6ea5", alpha=0.22, linewidth=0.9)
        ax.plot(range(1, len(vc) + 1), vc, color="#c0392b", alpha=0.22, linewidth=0.9)

    def mean_curve(curves):
        longest = max(len(c) for c in curves)
        padded = np.full((len(curves), longest), np.nan)
        for i, c in enumerate(curves):
            padded[i, :len(c)] = c
        return np.nanmean(padded, axis=0)

    mt, mv = mean_curve(train_curves), mean_curve(val_curves)
    ax.plot(range(1, len(mt) + 1), mt, color="#1f3f5f", linewidth=2.0, label="训练损失(折均值)")
    ax.plot(range(1, len(mv) + 1), mv, color="#8e1b10", linewidth=2.0, label="早停验证损失(折均值)")
    ax.set_xlabel("epoch")
    ax.set_ylabel("交叉熵损失")
    ax.set_title("最佳超参数组合的损失曲线（细线为各折自身曲线）")
    ax.xaxis.set_major_locator(MaxNLocator(integer=True))
    ax.legend()
```

```
    ax.grid(alpha=0.25)
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)


def plot_confusion(y_true, y_pred, path, title):
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(6.4, 4.5))
    im = ax.imshow(cm, cmap="Blues")
    ax.set_xticks([0, 1], CLASS_NAMES)
    ax.set_yticks([0, 1], CLASS_NAMES)
    ax.set_xlabel("预测标签")
    ax.set_ylabel("真实标签")
    ax.set_title(title, fontsize=10)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, str(cm[i, j]), ha="center", va="center", fontsize=13,
                    color="white" if cm[i, j] > cm.max() / 2 else "#20303f")
    fig.colorbar(im, ax=ax, label="样本数")
    fig.tight_layout()
    fig.savefig(path)
    plt.close(fig)
    return cm
```

6.2 程序完整运行输出

```
================================================================================
  SST-2 情感二分类：MLP 超参数网格搜索 + 重复 K 折交叉验证
================================================================================
  Python         : 3.12.12 (Linux)
  PyTorch        : 2.11.0+cu128
  计算设备       : cuda (NVIDIA GeForce RTX 4060 Laptop GPU)
  随机种子       : 42
  折数设置       : 5 折 x 2 次重复 = 10 个训练/验证划分

--------------------------------------------------------------------------------
  数据集
--------------------------------------------------------------------------------
  训练集: 67349 句, 标签分布 neg=29780 / pos=37569, 句长(词) min=1 max=52 均值=9.4
  测试集: 872 句, 标签分布 neg=428 / pos=444, 句长(词) min=2 max=47 均值=19.5

--------------------------------------------------------------------------------
  超参数搜索空间
--------------------------------------------------------------------------------
  hidden_size    : [64, 128, 256]
  dropout        : [0.0, 0.3, 0.5]
  lr             : [0.001, 0.005]
  weight_decay   : [0.0, 0.0001]
  组合数         : 36
  预计训练次数   : 36 x 10 = 360

--------------------------------------------------------------------------------
  构建交叉验证划分与折内特征
--------------------------------------------------------------------------------
  说明：TF-IDF 的词表与 IDF 只在本折训练集上拟合，再变换验证折；
        早停使用折内再切出的 10% 验证集，与外层评估折严格分离。
    [折  1/10] repeat=0 fold=0 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.53s
    [折  2/10] repeat=0 fold=1 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.53s
    [折  3/10] repeat=0 fold=2 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.59s
    [折  4/10] repeat=0 fold=3 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.48s
    [折  5/10] repeat=0 fold=4 内层训练=48492 早停验证=5388 外层验证=13469 词表=50000 非零特征/句=14.2 耗时=0.53s
    [折  6/10] repeat=1 fold=0 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.51s
    [折  7/10] repeat=1 fold=1 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.55s
```

```
    [折  8/10] repeat=1 fold=2 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.50s
    [折  9/10] repeat=1 fold=3 内层训练=48491 早停验证=5388 外层验证=13470 词表=50000 非零特征/句=14.2 耗时=0.60s
    [折 10/10] repeat=1 fold=4 内层训练=48492 早停验证=5388 外层验证=13469 词表=50000 非零特征/句=14.2 耗时=0.58s
  划分与特征构建总耗时 5.41s，外层验证样本合计 134698

--------------------------------------------------------------------------------
  串行与并行搜索耗时对比
--------------------------------------------------------------------------------
  在 8 个组合 x 5 折 上分别执行串行与并行搜索
  两支均固定 torch 线程数 = 2，因此加速比只反映进程级并行

--------------------------------------------------------------------------------
  >> 串行搜索：8 个组合 x 5 折（n_jobs=1）
--------------------------------------------------------------------------------
  [ 1/8] hidden=64   dropout=0.0  lr=0.001   wd=0        acc=0.9126 ± 0.0010  f1=0.9114  训练耗时=3.7s
  [ 2/8] hidden=64   dropout=0.0  lr=0.001   wd=0.0001   acc=0.9173 ± 0.0008  f1=0.9161  训练耗时=6.5s
  [ 3/8] hidden=64   dropout=0.0  lr=0.005   wd=0        acc=0.9078 ± 0.0014  f1=0.9065  训练耗时=1.5s
  [ 4/8] hidden=64   dropout=0.0  lr=0.005   wd=0.0001   acc=0.9183 ± 0.0011  f1=0.9172  训练耗时=5.2s
  [ 5/8] hidden=64   dropout=0.3  lr=0.001   wd=0        acc=0.9128 ± 0.0010  f1=0.9115  训练耗时=3.6s
  [ 6/8] hidden=64   dropout=0.3  lr=0.001   wd=0.0001   acc=0.9175 ± 0.0007  f1=0.9163  训练耗时=6.5s
  [ 7/8] hidden=64   dropout=0.3  lr=0.005   wd=0        acc=0.9114 ± 0.0022  f1=0.9101  训练耗时=1.6s
  [ 8/8] hidden=64   dropout=0.3  lr=0.005   wd=0.0001   acc=0.9189 ± 0.0011  f1=0.9178  训练耗时=5.4s
  串行搜索 完成，墙钟耗时 35.03s，累计训练耗时 34.0s

--------------------------------------------------------------------------------
  >> 并行搜索：8 个组合 x 5 折（n_jobs=4）
--------------------------------------------------------------------------------
  [ 1/8] hidden=64   dropout=0.0  lr=0.005   wd=0        acc=0.9078 ± 0.0014  f1=0.9065  训练耗时=4.8s
  [ 2/8] hidden=64   dropout=0.0  lr=0.001   wd=0        acc=0.9126 ± 0.0010  f1=0.9114  训练耗时=10.0s
  [ 3/8] hidden=64   dropout=0.3  lr=0.001   wd=0        acc=0.9128 ± 0.0010  f1=0.9115  训练耗时=11.0s
  [ 4/8] hidden=64   dropout=0.0  lr=0.005   wd=0.0001   acc=0.9183 ± 0.0011  f1=0.9172  训练耗时=16.5s
  [ 5/8] hidden=64   dropout=0.0  lr=0.001   wd=0.0001   acc=0.9173 ± 0.0008  f1=0.9161  训练耗时=19.9s
  [ 6/8] hidden=64   dropout=0.3  lr=0.005   wd=0        acc=0.9114 ± 0.0022  f1=0.9101  训练耗时=4.9s
  [ 7/8] hidden=64   dropout=0.3  lr=0.001   wd=0.0001   acc=0.9175 ± 0.0008  f1=0.9163  训练耗时=15.5s
  [ 8/8] hidden=64   dropout=0.3  lr=0.005   wd=0.0001   acc=0.9189 ± 0.0010  f1=0.9177  训练耗时=10.0s
  并行搜索 完成，墙钟耗时 30.03s，累计训练耗时 92.6s
  >> 加速比 1.17x (串行 35.03s -> 并行 30.03s, n_jobs=4)

--------------------------------------------------------------------------------
  网格搜索（完整搜索空间，并行执行）
```

```
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
  >> 完整网格搜索：36 个组合 x 10 折（n_jobs=4）
--------------------------------------------------------------------------------
  [ 1/36] hidden=64   dropout=0.0  lr=0.005   wd=0        acc=0.9075 ± 0.0014  f1=0.9061  训练耗时=9.1s
  [ 2/36] hidden=64   dropout=0.0  lr=0.001   wd=0        acc=0.9127 ± 0.0011  f1=0.9114  训练耗时=19.9s
  [ 3/36] hidden=64   dropout=0.3  lr=0.001   wd=0        acc=0.9131 ± 0.0012  f1=0.9118  训练耗时=22.2s
  [ 4/36] hidden=64   dropout=0.0  lr=0.005   wd=0.0001   acc=0.9178 ± 0.0012  f1=0.9166  训练耗时=31.4s
  [ 5/36] hidden=64   dropout=0.0  lr=0.001   wd=0.0001   acc=0.9168 ± 0.0014  f1=0.9155  训练耗时=39.3s
  [ 6/36] hidden=64   dropout=0.3  lr=0.005   wd=0        acc=0.9114 ± 0.0018  f1=0.9102  训练耗时=10.4s
  [ 7/36] hidden=64   dropout=0.3  lr=0.001   wd=0.0001   acc=0.9171 ± 0.0013  f1=0.9159  训练耗时=39.9s
  [ 8/36] hidden=64   dropout=0.5  lr=0.001   wd=0        acc=0.9130 ± 0.0012  f1=0.9117  训练耗时=24.1s
  [ 9/36] hidden=64   dropout=0.3  lr=0.005   wd=0.0001   acc=0.9186 ± 0.0012  f1=0.9174  训练耗时=33.5s
  [10/36] hidden=64   dropout=0.5  lr=0.005   wd=0        acc=0.9101 ± 0.0016  f1=0.9089  训练耗时=11.8s
  [11/36] hidden=64   dropout=0.5  lr=0.001   wd=0.0001   acc=0.9168 ± 0.0010  f1=0.9155  训练耗时=46.5s
  [12/36] hidden=128  dropout=0.0  lr=0.001   wd=0        acc=0.9129 ± 0.0011  f1=0.9116  训练耗时=27.3s
  [13/36] hidden=128  dropout=0.0  lr=0.005   wd=0        acc=0.9097 ± 0.0014  f1=0.9084  训练耗时=18.4s
  [14/36] hidden=64   dropout=0.5  lr=0.005   wd=0.0001   acc=0.9180 ± 0.0014  f1=0.9168  训练耗时=49.6s
  [15/36] hidden=128  dropout=0.3  lr=0.001   wd=0        acc=0.9123 ± 0.0009  f1=0.9111  训练耗时=39.4s
  [16/36] hidden=128  dropout=0.0  lr=0.001   wd=0.0001   acc=0.9175 ± 0.0015  f1=0.9162  训练耗时=81.1s
  [17/36] hidden=128  dropout=0.0  lr=0.005   wd=0.0001   acc=0.9175 ± 0.0015  f1=0.9163  训练耗时=68.3s
  [18/36] hidden=128  dropout=0.3  lr=0.005   wd=0        acc=0.9082 ± 0.0016  f1=0.9069  训练耗时=21.1s
  [19/36] hidden=128  dropout=0.3  lr=0.001   wd=0.0001   acc=0.9173 ± 0.0014  f1=0.9161  训练耗时=91.3s
  [20/36] hidden=128  dropout=0.5  lr=0.001   wd=0        acc=0.9124 ± 0.0011  f1=0.9111  训练耗时=43.2s
  [21/36] hidden=128  dropout=0.3  lr=0.005   wd=0.0001   acc=0.9185 ± 0.0014  f1=0.9173  训练耗时=67.5s
  [22/36] hidden=128  dropout=0.5  lr=0.005   wd=0        acc=0.9101 ± 0.0016  f1=0.9088  训练耗时=24.7s
  [23/36] hidden=256  dropout=0.0  lr=0.001   wd=0        acc=0.9123 ± 0.0011  f1=0.9110  训练耗时=51.5s
  [24/36] hidden=128  dropout=0.5  lr=0.001   wd=0.0001   acc=0.9176 ± 0.0012  f1=0.9164  训练耗时=106.4s
  [25/36] hidden=128  dropout=0.5  lr=0.005   wd=0.0001   acc=0.9190 ± 0.0014  f1=0.9178  训练耗时=91.0s
  [26/36] hidden=256  dropout=0.0  lr=0.005   wd=0        acc=0.8984 ± 0.0016  f1=0.8969  训练耗时=37.1s
  [27/36] hidden=256  dropout=0.3  lr=0.001   wd=0        acc=0.9123 ± 0.0014  f1=0.9110  训练耗时=74.0s
  [28/36] hidden=256  dropout=0.0  lr=0.001   wd=0.0001   acc=0.9183 ± 0.0016  f1=0.9171  训练耗时=184.3s
  [29/36] hidden=256  dropout=0.3  lr=0.005   wd=0        acc=0.9027 ± 0.0078  f1=0.9012  训练耗时=44.3s
  [30/36] hidden=256  dropout=0.0  lr=0.005   wd=0.0001   acc=0.9173 ± 0.0017  f1=0.9161  训练耗时=160.7s
  [31/36] hidden=256  dropout=0.5  lr=0.001   wd=0        acc=0.9124 ± 0.0012  f1=0.9112  训练耗时=80.0s
  [32/36] hidden=256  dropout=0.3  lr=0.001   wd=0.0001   acc=0.9180 ± 0.0013  f1=0.9168  训练耗时=198.4s
  [33/36] hidden=256  dropout=0.5  lr=0.005   wd=0        acc=0.9092 ± 0.0023  f1=0.9079  训练耗时=49.2s
  [34/36] hidden=256  dropout=0.3  lr=0.005   wd=0.0001   acc=0.9186 ± 0.0013  f1=0.9175  训练耗时=148.2s
  [35/36] hidden=256  dropout=0.5  lr=0.001   wd=0.0001   acc=0.9180 ± 0.0015  f1=0.9168  训练耗时=160.9s
```

```
  [36/36] hidden=256  dropout=0.5  lr=0.005   wd=0.0001   acc=0.9176 ± 0.0019  f1=0.9164  训练耗时=90.9s
  完整网格搜索 完成，墙钟耗时 602.22s，累计训练耗时 2296.7s

--------------------------------------------------------------------------------
  交叉验证结果排名（按平均准确率降序）
--------------------------------------------------------------------------------
    #   hidden  dropout       lr  weight_decay        准确率       标准差  F1(macro)   最优epoch
  ------------------------------------------------------------------------------------
    1      128      0.5    5e-03         1e-04     0.9190    0.0014     0.9178      17.4
    2      256      0.3    5e-03         1e-04     0.9186    0.0013     0.9175      17.5
    3       64      0.3    5e-03         1e-04     0.9186    0.0012     0.9174      20.1
    4      128      0.3    5e-03         1e-04     0.9185    0.0014     0.9173      17.2
    5      256      0.0    1e-03         1e-04     0.9183    0.0016     0.9171      27.5
    6      256      0.5    1e-03         1e-04     0.9180    0.0015     0.9168      26.5
    7       64      0.5    5e-03         1e-04     0.9180    0.0014     0.9168      20.4
    8      256      0.3    1e-03         1e-04     0.9180    0.0013     0.9168      27.2
    9       64      0.0    5e-03         1e-04     0.9178    0.0012     0.9166      19.0
   10      128      0.5    1e-03         1e-04     0.9176    0.0012     0.9164      29.2
   11      256      0.5    5e-03         1e-04     0.9176    0.0019     0.9164      15.3
   12      128      0.0    5e-03         1e-04     0.9175    0.0015     0.9163      18.4
   13      128      0.0    1e-03         1e-04     0.9175    0.0015     0.9162      28.4
   14      256      0.0    5e-03         1e-04     0.9173    0.0017     0.9161      19.5
   15      128      0.3    1e-03         1e-04     0.9173    0.0014     0.9161      29.2
   16       64      0.3    1e-03         1e-04     0.9171    0.0013     0.9159      29.9
   17       64      0.5    1e-03         1e-04     0.9168    0.0010     0.9155      29.6
   18       64      0.0    1e-03         1e-04     0.9168    0.0014     0.9155      29.8
   19       64      0.3    1e-03         0e+00     0.9131    0.0012     0.9118      11.9
   20       64      0.5    1e-03         0e+00     0.9130    0.0012     0.9117      13.1
   21      128      0.0    1e-03         0e+00     0.9129    0.0011     0.9116       7.5
   22       64      0.0    1e-03         0e+00     0.9127    0.0011     0.9114      10.4
   23      256      0.5    1e-03         0e+00     0.9124    0.0012     0.9112       6.7
   24      128      0.5    1e-03         0e+00     0.9124    0.0011     0.9111       9.2
   25      128      0.3    1e-03         0e+00     0.9123    0.0009     0.9111       8.2
   26      256      0.3    1e-03         0e+00     0.9123    0.0014     0.9110       5.9
   27      256      0.0    1e-03         0e+00     0.9123    0.0011     0.9110       5.2
   28       64      0.3    5e-03         0e+00     0.9114    0.0018     0.9102       2.9
   29       64      0.5    5e-03         0e+00     0.9101    0.0016     0.9089       3.0
   30      128      0.5    5e-03         0e+00     0.9101    0.0016     0.9088       2.7
   31      128      0.0    5e-03         0e+00     0.9097    0.0014     0.9084       2.0
   32      256      0.5    5e-03         0e+00     0.9092    0.0023     0.9079       2.2
```

```
   33      128      0.3    5e-03         0e+00     0.9082    0.0016     0.9069       2.0
   34       64      0.0    5e-03         0e+00     0.9075    0.0014     0.9061       2.0
   35      256      0.3    5e-03         0e+00     0.9027    0.0078     0.9012       1.5
   36      256      0.0    5e-03         0e+00     0.8984    0.0016     0.8969       1.0

--------------------------------------------------------------------------------
  最佳超参数组合
--------------------------------------------------------------------------------
  hidden=128  dropout=0.5  lr=0.005   wd=0.0001
  交叉验证平均准确率 : 0.9190 ± 0.0014
  交叉验证宏平均 F1  : 0.9178 ± 0.0015
  各折准确率         : [0.9195  0.9195  0.9172  0.9170  0.9203  0.9220  0.9179  0.9183  0.9197  0.9186]

  对照：最差组合 hidden=256  dropout=0.0  lr=0.005   wd=0
        交叉验证平均准确率 0.8984
        最佳组合相对最差组合提升 2.06 个百分点

--------------------------------------------------------------------------------
  >> 最终重训：hidden=128  dropout=0.5  lr=0.005   wd=0.0001 ，固定 epoch=17，训练集全量 67349 句
--------------------------------------------------------------------------------
  特征维度=50000 词表=50000 非零特征/句=14.1
    epoch   1/17  训练损失=0.4760
    epoch   2/17  训练损失=0.2235
    epoch   3/17  训练损失=0.1837
    epoch   4/17  训练损失=0.1683
    epoch   5/17  训练损失=0.1580
    epoch   6/17  训练损失=0.1498
    epoch   7/17  训练损失=0.1445
    epoch   8/17  训练损失=0.1427
    epoch   9/17  训练损失=0.1400
    epoch  10/17  训练损失=0.1353
    epoch  11/17  训练损失=0.1334
    epoch  12/17  训练损失=0.1326
    epoch  13/17  训练损失=0.1300
    epoch  14/17  训练损失=0.1289
    epoch  15/17  训练损失=0.1270
    epoch  16/17  训练损失=0.1256
    epoch  17/17  训练损失=0.1257
  重训耗时 2.0s
```

```
--------------------------------------------------------------------------------
  测试集评估（最佳组合，完整训练集重训）
--------------------------------------------------------------------------------

  测试集准确率 : 0.8096
  宏平均 Precision=0.8155  Recall=0.8083  F1=0.8083

              precision    recall  f1-score   support

    negative     0.8541    0.7383    0.7920       428
    positive     0.7769    0.8784    0.8245       444

    accuracy                         0.8096       872
   macro avg     0.8155    0.8083    0.8083       872
weighted avg     0.8148    0.8096    0.8086       872


--------------------------------------------------------------------------------
  可视化输出
--------------------------------------------------------------------------------
  已保存 figures/01_组合排名.png
  已保存 figures/02_top12_箱线图.png
  已保存 figures/03_hidden_dropout_热力图.png
  已保存 figures/04_学习率与权重衰减_边际影响.png
  已保存 figures/05_最佳组合损失曲线.png
  已保存 figures/06_测试集混淆矩阵.png

--------------------------------------------------------------------------------
  结果落盘
--------------------------------------------------------------------------------
  已保存 results/cv_summary.csv
  已保存 results/cv_per_fold.csv
  已保存 results/best_model.pt
  已保存 results/best_params.json

================================================================================
  完成
================================================================================
  完整网格搜索墙钟耗时 : 602.22s
  程序总耗时           : 676.36s
```

6.3 辅助脚本：稀疏输入喂法效率基准（4.6 节）

```
"""稀疏输入喂法的效率基准与批量大小敏感性实验（报告 4.6 节）。

比较 COO / CSR / 稠密三种喂法单轮耗时，拆分「取批数据」与「前向反向」的
开销，并考察批量大小对准确率的影响。
用法：.venv/bin/python -u lab1/efficiency_bench.py
"""

import argparse
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from scipy import sparse

import sys

sys.path.insert(0, str(Path(__file__).resolve().parent))
from hs_core import banner, pick_device, silence_sparse_warnings
from hs_data import build_folds, load_sst2
from hs_model import sparse_batch, train_fold

ROOT = Path(__file__).resolve().parent.parent

# 基准用的折规模与主程序一致：5 折下每折约 48491 句训练、特征维 50000
DEFAULT_N = 48491
DEFAULT_DIM = 50000
DEFAULT_NNZ = 14


class BenchNet(nn.Module):
    """与主程序同构的单隐层 MLP，但输入格式由调用方决定。"""

    def __init__(self, input_dim, hidden):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden)
        self.fc2 = nn.Linear(hidden, 2)
```

```
    def forward(self, x):
        if x.is_sparse:
            h = torch.sparse.mm(x, self.fc1.weight.t()) + self.fc1.bias
        else:
            h = self.fc1(x)
        return self.fc2(torch.relu(h))


def synth_fold(n, dim, nnz, seed=0):
    """造一份与真实折同规模、同稀疏度的 CSR 特征矩阵。

    不读真实数据是为了让基准只反映「喂法」的差异，不被 TF-IDF 拟合时间干扰；
    稀疏度与维度都按实测值设定，因此单次矩阵乘的规模与真实情形一致。
    """
    rng = np.random.default_rng(seed)
    rows = np.repeat(np.arange(n), nnz)
    cols = rng.integers(0, dim, size=rows.size)
    vals = rng.random(rows.size).astype(np.float32)
    return sparse.csr_matrix((vals, (rows, cols)), shape=(n, dim), dtype=np.float32)


def make_feeders(csr, device):
    """构造三种「取一批数据」的实现，返回同一个批次张量。"""

    def coo(idx):
        sub = csr[idx]
        c = sub.tocoo()
        return torch.sparse_coo_tensor(
            torch.from_numpy(np.vstack([c.row, c.col]).astype(np.int64)),
            torch.from_numpy(c.data.astype(np.float32, copy=False)),
            size=c.shape,
        ).to(device)

    def csr_fmt(idx):
        return sparse_batch(csr, idx, device)

    def dense(idx):
        sub = csr[idx]
        return torch.from_numpy(sub.toarray()).to(device)
```

```
    return {"COO": coo, "CSR": csr_fmt, "稠密": dense}


def train_one_epoch(net, optimizer, criterion, fetch, y_all, n, bs, perm, device):
    """跑一轮：按给定顺序遍历全部数据。"""
    for start in range(0, (n // bs) * bs, bs):
        idx = perm[start:start + bs]
        xb = fetch(idx)
        yb = y_all[torch.from_numpy(idx).to(device)]
        loss = criterion(net(xb), yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()


def time_epochs(net, fetch, n, bs, device, epochs=3, seed=0):
    optimizer = torch.optim.Adam(net.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss()
    y_all = torch.randint(0, 2, (n,), device=device)
    d = torch.device(device)

    # 预热一次，排除首次显存分配与算子编译的影响
    train_one_epoch(net, optimizer, criterion, fetch, y_all, n, bs,
                    np.random.default_rng(seed).permutation(n), d)
    if torch.cuda.is_available():
        torch.cuda.synchronize()

    t0 = time.perf_counter()
    for e in range(epochs):
        perm = np.random.default_rng(seed + e + 1).permutation(n)
        train_one_epoch(net, optimizer, criterion, fetch, y_all, n, bs, perm, d)
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    return (time.perf_counter() - t0) / epochs * 1000, n // bs


def bench_formats(n, dim, nnz, device, batch_sizes, hidden):
    print("\n" + "=" * 78)
    print(f"  一、稀疏输入格式与批量大小对单轮训练耗时的影响（hidden={hidden}）")
```

```
    print("=" * 78)
    print(f"  折规模 {n} 句 x {dim} 维，非零特征/句={nnz}，"
          f"设备={device}，第一层参数 {dim * hidden / 1e6:.1f}M")

    csr = synth_fold(n, dim, nnz)
    rows = []
    for bs in batch_sizes:
        feeders = make_feeders(csr, device)
        for name, fetch in feeders.items():
            net = BenchNet(dim, hidden).to(device)
            ms, iters = time_epochs(net, fetch, n, bs, device)
            rows.append((name, bs, ms, iters))
            del net
            if torch.cuda.is_available():
                torch.cuda.empty_cache()

    base = rows[0][2]
    print(f"\n  {'格式':<6}{'批量':>7}{'每轮迭代数':>11}{'每轮耗时':>12}{'单次迭代':>11}{'相对':>8}")
    print("  " + "-" * 56)
    for name, bs, ms, iters in rows:
        print(f"  {name:<6}{bs:>7}{iters:>11}{ms:>10.1f}ms"
              f"{ms / iters:>9.2f}ms{base / ms:>7.2f}x")
    return rows


def bench_components(n, dim, nnz, device, bs, hidden):
    print("\n" + "=" * 78)
    print(f"  二、单次迭代的耗时构成（hidden={hidden}，批量={bs}）")
    print("=" * 78)
    csr = synth_fold(n, dim, nnz)
    net = BenchNet(dim, hidden).to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss()
    y_all = torch.randint(0, 2, (n,), device=device)
    perm = np.random.default_rng(0).permutation(n)
    iters = n // bs

    def slice_only():
        for s in range(0, iters * bs, bs):
            csr[perm[s:s + bs]].tocoo()
```

```
def fetch_only():
    for s in range(0, iters * bs, bs):
        sparse_batch(csr, perm[s:s + bs], device)

batches = [sparse_batch(csr, perm[s:s + bs], device)
           for s in range(0, iters * bs, bs)]

def step_only():
    for b in batches:
        loss = criterion(net(b), y_all[:bs])
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

def full():
    for s in range(0, iters * bs, bs):
        b = sparse_batch(csr, perm[s:s + bs], device)
        loss = criterion(net(b), y_all[:bs])
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

def timeit(fn, reps=3):
    fn()
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(reps):
        fn()
    if torch.cuda.is_available():
        torch.cuda.synchronize()
    return (time.perf_counter() - t0) / reps / iters * 1000

print(f"  批量={bs}，每轮 {iters} 次迭代")
parts = [
    ("scipy 行切片（纯 CPU）", timeit(slice_only)),
    ("取一批数据（切片+转 CSR+搬运）", timeit(fetch_only)),
    ("模型步（前向+反向+参数更新）", timeit(step_only)),
    ("两者合计（真实训练）", timeit(full)),
```

```
    ]
    for label, ms in parts:
        print(f"    {label:<30}{ms:>8.3f} ms/迭代")
    return parts


def probe_batch_accuracy(args, batch_sizes, lrs):
    """在单个折上比较不同批量大小的验证准确率与耗时。"""
    print("\n" + "=" * 78)
    print("  三、批量大小敏感性：准确率与耗时")
    print("=" * 78)
    texts, labels = load_sst2(args.data_dir / "train.tsv")
    cfg = {
        "seed": 42, "epochs": args.epochs, "patience": 5,
        "n_splits": args.n_splits, "n_repeats": 1,
        "torch_threads": args.torch_threads, "device": args.device,
        "batch_size": batch_sizes[0],
        "tfidf": {"ngram_range": (1, 2), "min_df": args.min_df,
                  "max_features": args.max_features, "sublinear_tf": True},
    }
    folds = build_folds(texts, labels, cfg)
    fold = folds[0]
    device = pick_device(args.device)

    print(f"\n  在折 0 上固定 hidden=256 dropout=0.0 weight_decay=0.0，"
          f"最大 {args.epochs} 轮，早停 patience=5")
    print(f"\n  {'批量':>7}{'学习率':>9}{'最优轮':>8}{'验证准确率':>12}"
          f"{'每折耗时':>11}{'相对耗时':>10}")
    print("  " + "-" * 57)
    ref = None
    for bs in batch_sizes:
        for lr in lrs:
            c = dict(cfg, batch_size=bs)
            combo = {"hidden_size": 256, "dropout": 0.0,
                     "lr": lr, "weight_decay": 0.0}
            r = train_fold(combo, fold, c, device)
            if ref is None:
                ref = r["train_seconds"]
            print(f"  {bs:>7}{lr:>9.0e}{r['best_epoch']:>8}{r['accuracy']:>12.4f}"
                  f"{r['train_seconds']:>10.1f}s{ref / r['train_seconds']:>9.2f}x")
```

```
def parse_args():
    p = argparse.ArgumentParser(description="稀疏输入喂法的效率基准")
    p.add_argument("--data-dir", type=Path, default=ROOT / "demo_labs" / "SST-2")
    p.add_argument("--device", default="auto")
    p.add_argument("--n", type=int, default=DEFAULT_N)
    p.add_argument("--dim", type=int, default=DEFAULT_DIM)
    p.add_argument("--nnz", type=int, default=DEFAULT_NNZ)
    p.add_argument("--max-features", type=int, default=50000)
    p.add_argument("--min-df", type=int, default=3)
    p.add_argument("--n-splits", type=int, default=5)
    p.add_argument("--epochs", type=int, default=30)
    p.add_argument("--torch-threads", type=int, default=8)
    p.add_argument("--hidden", type=int, default=64,
                   help="第一层隐藏单元数，决定第一层参数量")
    p.add_argument("--only", choices=["formats", "components", "probe"],
                   help="只跑其中一节")
    p.add_argument("--skip-probe", action="store_true",
                   help="跳过会真正读数据的批量大小敏感性实验")
    return p.parse_args()


def main():
    args = parse_args()
    silence_sparse_warnings()
    torch.set_num_threads(args.torch_threads)
    device = "cuda" if (args.device == "auto" and torch.cuda.is_available()) \
        else args.device
    if device == "auto":
        device = "cpu"

    banner("稀疏输入喂法效率基准")
    print(f"  计算设备      : {device}"
          + (f" ({torch.cuda.get_device_name(0)})" if device == "cuda" else ""))
    print(f"  CPU 线程数    : {args.torch_threads}")

    batch_sizes = [256, 512, 1024, 2048, 4096]
    if args.only in (None, "formats"):
        bench_formats(args.n, args.dim, args.nnz, device, batch_sizes, args.hidden)
```

```
    if args.only in (None, "components"):
        bench_components(args.n, args.dim, args.nnz, device, 2048, args.hidden)
    if args.only in (None, "probe") and not args.skip_probe:
        probe_batch_accuracy(args, [256, 512, 1024, 2048], [1e-3, 5e-3])

    if torch.cuda.is_available():
        print(f"\n  显存峰值      : "
              f"{torch.cuda.max_memory_allocated() / 2**20:.0f} MiB")
    print("\n  完成。")


if __name__ == "__main__":
    main()
```

6.4 辅助脚本：特征表示消融对照（4.5.3 节）

```
"""特征表示消融对照（报告 4.5 节）。

在同一个 SST-2 划分上比较「TF-IDF 先经 SVD 降维再喂稠密模型」与「TF-IDF
稀疏直接接入」两条路线，量化降维的损失；并给出各路线交叉验证分数与独立
测试集分数的落差，说明前者为何不能直接当作后者的预期。
用法：.venv/bin/python -u lab1/ablation_features.py
"""

import argparse
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from sklearn.decomposition import TruncatedSVD
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.preprocessing import StandardScaler

import sys

sys.path.insert(0, str(Path(__file__).resolve().parent))
from hs_core import banner, pick_device, silence_sparse_warnings
from hs_data import build_feature_matrices, load_sst2

ROOT = Path(__file__).resolve().parent.parent


def make_dense_model(input_dim, hidden=256, dropout=0.3, seed=42):
    torch.manual_seed(seed)
    return nn.Sequential(
        nn.Linear(input_dim, hidden),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(hidden, 2),
    )
```

```
def train_dense_mlp(X_tr, y_tr, X_te, cfg, seed=42):
    """在稠密特征上训练单隐层 MLP，返回 (验证准确率, 测试准确率)。

    训练集内部再切 10% 作早停集，与主程序口径一致。
    """
    device = pick_device(cfg["device"])
    torch.manual_seed(seed)
    rng = np.random.default_rng(seed)
    n = X_tr.shape[0]
    idx = rng.permutation(n)
    n_val = int(n * 0.1)
    inner, early = idx[n_val:], idx[:n_val]

    model = make_dense_model(X_tr.shape[1], seed=seed).to(device)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    crit = nn.CrossEntropyLoss()

    X_inner = torch.from_numpy(X_tr[inner]).to(device)
    y_inner = torch.from_numpy(y_tr[inner]).long().to(device)
    X_early = torch.from_numpy(X_tr[early]).to(device)
    y_early = torch.from_numpy(y_tr[early]).long().to(device)
    X_test = torch.from_numpy(X_te).to(device)

    best_loss, best_state = float("inf"), None
    bs = 2048
    for epoch in range(1, cfg["epochs"] + 1):
        model.train()
        perm = np.random.default_rng(seed + epoch).permutation(len(inner))
        for s in range(0, len(inner), bs):
            b = perm[s:s + bs]
            loss = crit(model(X_inner[b]), y_inner[b])
            opt.zero_grad()
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            vl = crit(model(X_early), y_early).item()
        if vl < best_loss - 1e-5:
            best_loss = vl
```

```
            best_state = {k: v.detach().clone()
                          for k, v in model.state_dict().items()}
        elif epoch - 1 >= cfg["patience"]:
            break
    if best_state is not None:
        model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        test_acc = (model(X_test).argmax(1).cpu().numpy() == y_te).mean()
    return float(test_acc)


def evaluate(name, X_all, y_all, X_test, y_test, cfg, use_mlp,
             extra_prep=None):
    """跑重复分层 K 折得到交叉验证分数，再在测试集上评估。"""
    splitter = RepeatedStratifiedKFold(
        n_splits=cfg["n_splits"], n_repeats=cfg["n_repeats"],
        random_state=cfg["seed"],
    )
    accs, t0 = [], time.time()
    for tr, va in splitter.split(X_all, y_all):
        X_tr, X_va = X_all[tr], X_all[va]
        y_tr, y_va = y_all[tr], y_all[va]
        if extra_prep is not None:
            X_tr, X_va = extra_prep(X_tr, X_va)
        if use_mlp:
            model = make_dense_model(X_tr.shape[1], seed=cfg["seed"]).to(
                pick_device(cfg["device"]))
            opt = torch.optim.Adam(model.parameters(), lr=1e-3)
            crit = nn.CrossEntropyLoss()
            Xt = torch.from_numpy(X_tr).to(pick_device(cfg["device"]))
            yt = torch.from_numpy(y_tr).long().to(pick_device(cfg["device"]))
            bs = 2048
            for _ in range(cfg["epochs"]):
                perm = np.random.default_rng(cfg["seed"]).permutation(len(tr))
                for s in range(0, len(tr), bs):
                    b = perm[s:s + bs]
                    loss = crit(model(Xt[b]), yt[b])
                    opt.zero_grad()
                    loss.backward()
```

```
                    opt.step()
            model.eval()
            with torch.no_grad():
                pred = model(torch.from_numpy(X_va).to(
                    pick_device(cfg["device"]))).argmax(1).cpu().numpy()
        else:
            clf = LogisticRegression(max_iter=1000)
            clf.fit(X_tr, y_tr)
            pred = clf.predict(X_va)
        accs.append((pred == y_va).mean())

    # 测试集：在完整训练集上重训
    if extra_prep is not None:
        X_tr_full, X_te = extra_prep(X_all, X_test)
    else:
        X_tr_full, X_te = X_all, X_test
    if use_mlp:
        test_acc = train_dense_mlp(X_tr_full, y_all, X_te, cfg)
    else:
        clf = LogisticRegression(max_iter=1000)
        clf.fit(X_tr_full, y_all)
        test_acc = float((clf.predict(X_te) == y_test).mean())

    cv = float(np.mean(accs))
    print(f"  {name:<26} CV={cv:.4f}±{np.std(accs):.4f}  "
          f"TEST={test_acc:.4f}  落差={cv - test_acc:+.4f}  "
          f"耗时={time.time() - t0:.0f}s")
    return {"name": name, "cv": cv, "cv_std": float(np.std(accs)),
            "test": test_acc, "gap": cv - test_acc}


def parse_args():
    p = argparse.ArgumentParser(description="特征表示消融对照")
    p.add_argument("--data-dir", type=Path, default=ROOT / "demo_labs" / "SST-2")
    p.add_argument("--device", default="auto")
    p.add_argument("--n-splits", type=int, default=5)
    p.add_argument("--n-repeats", type=int, default=2)
    p.add_argument("--epochs", type=int, default=30)
    p.add_argument("--patience", type=int, default=5)
    p.add_argument("--max-features", type=int, default=50000)
```

```
    p.add_argument("--min-df", type=int, default=3)
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--torch-threads", type=int, default=8)
    return p.parse_args()


def main():
    args = parse_args()
    silence_sparse_warnings()
    torch.set_num_threads(args.torch_threads)
    cfg = {
        "seed": args.seed, "epochs": args.epochs, "patience": args.patience,
        "device": args.device, "n_splits": args.n_splits,
        "n_repeats": args.n_repeats,
    }

    banner("特征表示消融对照：稀疏 TF-IDF vs SVD 降维")
    train_texts, y_train = load_sst2(args.data_dir / "train.tsv")
    test_texts, y_test = load_sst2(args.data_dir / "test.tsv")
    y_train, y_test = np.asarray(y_train), np.asarray(y_test)

    print("  在完整训练集上拟合 TF-IDF（仅用于本次对照，故不放在折内）")
    vec_cfg = {"ngram_range": (1, 2), "min_df": args.min_df,
               "max_features": args.max_features, "sublinear_tf": True}
    X_train, X_test, _ = build_feature_matrices(
        train_texts, test_texts, {"tfidf": vec_cfg})
    print(f"  原始稀疏特征: train={X_train.shape} test={X_test.shape} "
          f"nnz/句={X_train.nnz / X_train.shape[0]:.1f}")

    def svd_prep(k):
        def prep(X_tr, X_ap):
            svd = TruncatedSVD(n_components=k, random_state=args.seed)
            Z_tr = svd.fit_transform(X_tr).astype(np.float32)
            Z_ap = svd.transform(X_ap).astype(np.float32)
            sc = StandardScaler()
            return (sc.fit_transform(Z_tr).astype(np.float32),
                    sc.transform(Z_ap).astype(np.float32))
        return prep

    print("\n  对照组（每条路线都做 5 折 x 2 次重复交叉验证 + 独立测试集）")
```

```
    print("  " + "-" * 74)
    rows = []
    rows.append(evaluate("TF-IDF 稀疏 + LR", X_train, y_train, X_test, y_test,
                         cfg, use_mlp=False))

    # 稀疏 + MLP：复用主程序的口径，这里只报随机搜索出来的最佳组合结果
    print("  TF-IDF 稀疏 + MLP           见主程序 4.4 节（CV=0.9190 TEST=0.8096）")

    for k in (200, 800):
        rows.append(evaluate(f"TF-IDF + SVD{k} + LR", X_train, y_train,
                             X_test, y_test, cfg, use_mlp=False,
                             extra_prep=svd_prep(k)))
        svd = TruncatedSVD(n_components=k, random_state=args.seed)
        svd.fit(X_train)
        ev = svd.explained_variance_ratio_.sum()
        print(f"       （SVD{k} 累计解释方差 = {ev:.3f}，即原稀疏特征的 "
              f"{ev * 100:.1f}%）")

    print("\n  结论：SVD 降维后的稠密特征在本任务上远不如原始稀疏特征；"
          "\n        降维丢掉的正是 TF-IDF 里区分度最高的稀有词与二元词组。")
    print("\n  完成。")


if __name__ == "__main__":
    main()
```

7 参考文献

1. 课程组. 《深度学习》实验报告要求与实验一说明. 2026.
2. 课程示例程序 `demo_labs/lab1-hyper-paramter.py` 及数据集 `demo_labs/SST-2/`.
3. Socher R, Perelygin A, Wu J, et al. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. EMNLP, 2013.（SST-2 数据集出处）
4. Kohavi R. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. IJCAI, 1995.（K 折交叉验证）
5. Bergstra J, Bengio Y. Random Search for Hyper-Parameter Optimization. JMLR, 2012.（网格搜索与随机搜索的代价对比）
6. Pedregosa F, et al. Scikit-learn: Machine Learning in Python. JMLR, 2011.（`sklearn` 实现）
7. Paszke A, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS, 2019.
8. 稀疏张量与稀疏矩阵乘法接口：`torch.sparse_coo_tensor`、`torch.sparse_csr_tensor`、`torch.sparse.mm`. PyTorch 官方文档.
9. 并行搜索后端：`joblib.Parallel` 与 loky 后端. joblib 官方文档.

10. Loshchilov I, Hutter F. Decoupled Weight Decay Regularization. ICLR, 2019.（权重衰减对训练稳定性的作用）